

DOCUMENTATION TECHNIQUE

Projet IPBX / VoIP sous Asterisk

IVR · Routage · CDR · Facturation prepayee/postpayee · Conference

Asterisk 22.5.2 · PostgreSQL 18 · Ubuntu Server 24.04 LTS

Documentation technique — Projet IPBX/VoIP sous Asterisk

Table des matières

1. [Méthodologie professionnelle de réalisation et d'apprentissage du projet](#)
 2. [Présentation du projet](#)
 3. [Prérequis](#)
 4. [Installation complète](#)
 5. [Toutes les commandes exécutées](#)
 6. [Configuration](#)
 7. [Chronologie complète du projet](#)
 8. [Toutes les difficultés rencontrées](#)
 9. [Toutes les erreurs rencontrées](#)
 10. [Toutes les corrections apportées](#)
 11. [Décisions techniques](#)
 12. [Arborescence finale](#)
 13. [Scripts](#)
 14. [Vérifications](#)
 15. [Tests réalisés](#)
 16. [Conseils et bonnes pratiques](#)
 17. [Mise à jour — Session complémentaire \(repli d'agents, extensions directes, sécurisation\)](#)
 18. [Mise à jour — Session 3 \(contrôle de crédit temps réel, fiabilisation démarrage, console web, HTTPS\)](#)
 19. [Conclusion](#)
-

0. Méthodologie professionnelle de réalisation et d'apprentissage du projet

Cette section décrit **la méthode de travail** suivie pour ce projet — pas seulement le résultat technique. Elle sert à la fois de guide si le projet devait être refait ou repris par quelqu'un d'autre, et de préparation directe pour l'expliquer à l'oral.

Légende de statut utilisée dans tout le document

À partir de maintenant, chaque étape ou fonctionnalité du document est marquée avec l'un de ces quatre statuts, pour ne jamais prétendre qu'une chose fonctionne si elle n'a pas été réellement vérifiée :

Statut	Signification
[FAIT]	Réalisé et vérifié par un test concret
[PARTIEL]	Configuré, code écrit, mais pas encore testé en conditions réelles
[NON FAIT]	Non réalisé
[EN COURS]	Problème identifié, en cours de résolution

1. Commencer par comprendre le besoin

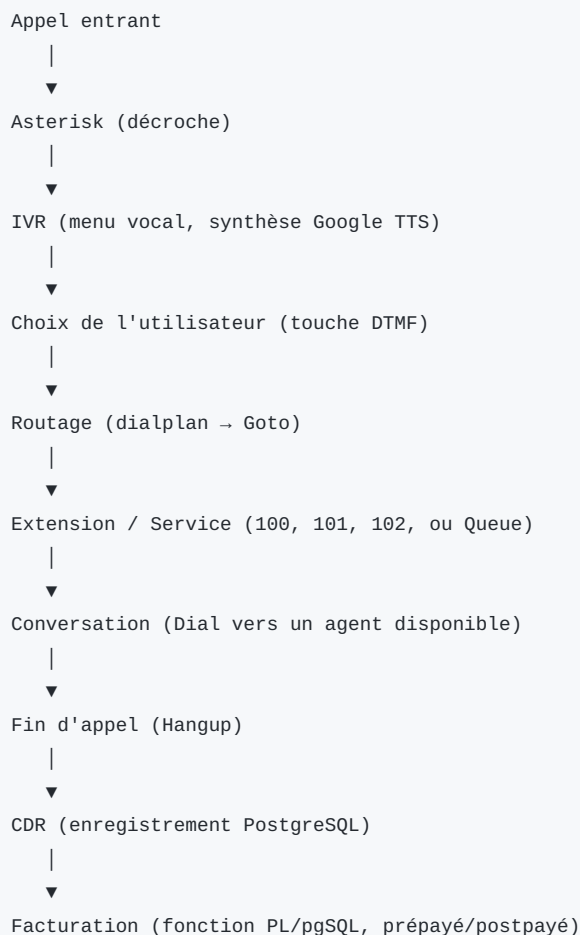
Avant de taper la moindre commande, reformuler le projet en une seule phrase simple :

“Construire un système téléphonique capable de recevoir un appel, présenter un menu vocal (IVR), orienter l'appel vers le bon service ou agent disponible, enregistrer les informations de chaque appel, et facturer l'appelant selon un mode prépayé ou postpayé.”

Toute décision technique prise ensuite doit pouvoir se rattacher à cette phrase — si une étape ne sert ni à recevoir, ni à orienter, ni à enregistrer, ni à facturer l'appel, il faut se demander pourquoi elle est là.

2. Découper le projet en petits blocs

Le projet complet se décompose en une seule chaîne logique, dans laquelle chaque flèche est une vraie dépendance technique :



Chaque bloc de ce document (sections 3 à 14) correspond à l'un de ces maillons.

3. Comprendre chaque bloc avant de le configurer

Pour chaque brique technique du projet, la question à se poser systématiquement, **avant** d'écrire une seule ligne de configuration :

- Quel est son rôle exact ?
- Pourquoi est-elle nécessaire (que se passerait-il sans elle) ?
- Avec quels autres composants communique-t-elle, et par quel moyen (fichier de config, socket réseau, appel système) ?
- Quels fichiers et quelles technologies sont concernés ?
- Est-ce que je pourrais l'expliquer à l'oral sans notes ?

Exemple appliqué à Asterisk lui-même : son rôle est de décrocher les appels SIP et d'exécuter le dialplan ; il est nécessaire car aucun softphone ne peut se parler directement sans un intermédiaire qui route les appels ; il communique avec PJSIP (comptes SIP), avec PostgreSQL (via le module `cdr_pgsq1` et des scripts `System()`), et avec des scripts externes (AGI) ; les fichiers concernés sont `pjsip_custom.conf` , `extensions_custom.conf` , `cdr_pgsq1.conf` .

4. Construire progressivement

Le projet n'a jamais été développé "d'un bloc" — chaque brique n'a été ajoutée qu'une fois la précédente validée par un test réel. Statut réel, à ce jour, de chaque grande étape :

Étape	Statut
Asterisk installé et fonctionnel	[FAIT]
Communication entre deux softphones (6001 ↔ 6002)	[FAIT]
IVR fonctionnel avec synthèse vocale	[FAIT]
Routage vers les 3 services (100, 101, 102)	[FAIT]
Mise en attente (Hold)	[FAIT]
Transfert d'appel (option <code>tt</code>)	[PARTIEL] (activé dans la config, jamais testé explicitement de bout en bout)
Conférence via numéro dédié (9000, <code>ConfBridge</code>)	[FAIT] (validé à 3 participants)
Conférence via fusion d'appels côté softphone (Add call + CONF)	[NON FAIT] (expliqué, pas encore testé)
CDR (enregistrement PostgreSQL)	[FAIT]
Base de facturation séparée (<code>a2billing</code>) + liaison <code>postgres_fdw</code>	[FAIT]
Facturation automatique prépayée	[FAIT]
Facturation automatique postpayée	[FAIT]
Codes de recharge (génération manuelle, usage, suppression)	[FAIT]
Recharge par composition téléphonique directe (<code>#XXX#</code>)	[FAIT]
Correction anti-injection SQL (variables <code>psql -v</code> , via <code>echo \\\ psql</code> — voir remarque)	[FAIT] (réappliqué et testé : <code>facturer.sh</code> et <code>recharger.sh</code> validés séparément)
Repli automatique vers un second agent si le premier est occupé (<code>queue</code> , <code>ringall</code>)	[FAIT] (validé : agent occupé exclu, mise en attente si tous occupés, sonnerie dès qu'un agent se libère)
Stratégie <code>leastrecent</code> (répartition équitable)	[NON FAIT] (expliquée en détail, jamais appliquée ni testée — la configuration active reste <code>ringall</code>)
Extensions d'appel direct entre comptes (6001, 6002, 6003)	[FAIT]
Contrôle de crédit prépayé en temps réel (blocage si solde ≤ 0 avant l'appel)	[FAIT] (testé : solde à 0 → appel refusé avec message vocal, aucune sonnerie côté agents ; solde repositionné → appel abouti normalement)
Fiabilisation de l'ordre de démarrage Asterisk/PostgreSQL (<code>systemd override</code>)	[FAIT] (testé après redémarrage complet de la VM : module <code>cdr_pgsql</code> actif dès le démarrage, sans rechargement manuel)
Console web de supervision (comptes + CDR en quasi temps réel)	[FAIT] (installée, testée, affiche correctement les appels et soldes)
HTTPS + nom de domaine local pour la console web	

Étape	Statut
	[FAIT] (testé : <code>https://voip.local</code> accessible, avertissement de certificat auto-signé accepté manuellement, connexion chiffrée fonctionnelle)

5. Toujours vérifier après chaque étape

Pour chaque étape du projet, le document (sections 13 et 14) précise systématiquement : ce qui doit fonctionner, la commande de test exacte, le résultat attendu, et la conduite à tenir si le résultat diffère. C'est cette discipline — ne jamais passer à la brique suivante sans avoir vérifié la précédente — qui a permis d'isoler chaque bug rapidement plutôt que de devoir chercher parmi plusieurs causes possibles à la fois.

6. Comprendre les commandes au lieu de les copier

Exemple appliqué à une ligne centrale du projet :

```
Dial(PJSIP/6002,20,tT)
```

Élément	Rôle
<code>Dial(...)</code>	Application Asterisk qui lance un appel sortant vers une ou plusieurs destinations
<code>PJSIP/</code>	Précise le canal/protocole à utiliser — ici PJSIP, l'implémentation moderne de SIP dans Asterisk (par opposition à l'ancien <code>chan_sip</code> , obsolète)
<code>6002</code>	L'identifiant de l'endpoint (compte SIP) à appeler, tel que déclaré dans <code>pjsip_custom.conf</code>
<code>20</code>	Délai de sonnerie maximal en secondes avant abandon
<code>tT</code>	Options : autorise l'appelé (<code>t</code>) et l'appelant (<code>T</code>) à transférer l'appel en cours de communication

Rôle global de la ligne : *“lance un appel vers le compte 6002, laisse sonner 20 secondes maximum, et autorise les deux interlocuteurs à transférer l'appel une fois connectés.”* C'est cette décomposition, systématiquement appliquée à chaque commande du projet, qui permet de reconstruire n'importe quelle ligne de configuration sans la mémoriser par cœur — et de l'expliquer clairement devant un professeur.

7. Schémas d'architecture

Flux d'un appel entrant vers l'IVR :

```
Softphone (SIP) → Asterisk → IVR (googletts.agi) → Dialplan → Extension/Service
```

Flux d'enregistrement des appels :

```
Asterisk → module cdr_pgsql → PostgreSQL (base asterisk_cdr, table cdr)
```

Flux de facturation :

```
Asterisk (Hangup Handler) → script facturer.sh → PostgreSQL (base a2billing)
|
fonction facturer_appel()
|
solde (prépayé) OU dette (postpayé)
```

```
fonction facturer_appel()
```

solde (prépayé) OU dette (postpayé)

Flux de liaison entre les deux bases :

```
Base a2billing —(postgres_fdw)→ Base asteriskcdr
      |                               |
      table cdr_distant (miroir, lecture seule) — table cdr (originale)
```

```
table cdr_distant (miroir, lecture seule) — table cdr (originale)
```

8. Journal de projet — modèle et exemple

Chaque séance de travail devrait, dans l'idéal, être notée selon ce modèle :

Champ	Contenu
Date	Date de la séance
Objectif	Ce qu'on cherchait à accomplir
Actions réalisées	Liste des étapes suivies
Commandes utilisées	Commandes exactes exécutées
Résultat obtenu	Ce qui s'est réellement passé
Erreur rencontrée	Message d'erreur exact, si applicable
Cause	Cause identifiée après diagnostic
Solution	Ce qui a corrigé le problème
Ce qui a été appris	Concept ou réflexe retenu pour la suite

Exemple réel tiré de ce projet (séance sur le repli d'agent occupé) :

Champ	Contenu
Objectif	Faire sonner un second agent (6003) si le premier (6002) est occupé
Actions réalisées	Ajout de <code>device_state_busy_at=1</code> sur les endpoints ; création de <code>queues.conf</code> avec stratégie <code>ringall</code> ; remplacement de <code>Dial()</code> par <code>Queue()</code> dans le dialplan
Commandes utilisées	<pre>sudo asterisk -rx "queue show service_client", sudo asterisk -rx "core show channels", watch -n 2 'sudo asterisk -rx "core show channels"'</pre>
Résultat obtenu (1 ^{er} essai)	6002 reste marqué <code>Not in use</code> alors qu'il est en communication active
Erreur rencontrée	Aucun message d'erreur — comportement silencieusement incorrect (pas de repli observé)
Cause	Le premier test était mal isolé : c'était 6002 lui-même qui initiait l'appel testé, et par ailleurs <code>ringinuse</code> n'était pas encore désactivé dans <code>queues.conf</code>
Solution	Ajout de <code>ringinuse = no</code> dans <code>queues.conf</code> ; protocole de test corrigé pour bien séparer "l'agent qu'on occupe" de "la personne qui teste" ; usage de <code>watch</code> pour observer les canaux en direct sans décalage de timing
Ce qui a été appris	Isoler une seule variable à la fois lors d'un test ; ne jamais interpréter un silence de logs comme une absence de logique testée — toujours vérifier l'état en direct pendant l'action, pas après coup

9. Méthode de dépannage

Règle centrale suivie tout au long du projet : **ne jamais modifier plusieurs éléments à la fois**. À chaque anomalie :

1. **Identifier le symptôme** précisément (message exact, comportement observé, pas une impression)
2. **Formuler une hypothèse** unique et vérifiable
3. **Vérifier cette hypothèse** avec une commande de diagnostic ciblée, sans encore rien changer
4. **Modifier un seul élément** à la fois
5. **Refaire le test**, dans les mêmes conditions que la première observation
6. **Confirmer ou éliminer l'hypothèse** selon le résultat
7. **Documenter la solution**, y compris si l'hypothèse était fausse — savoir ce qui n'était *pas* la cause a aussi de la valeur

Cette méthode est celle qui a permis, par exemple, de distinguer un vrai bug de configuration (Bug #7, double section `[global]` dans `cdr_pgsql.conf`) d'un simple problème de timing de test (le cas du terminal `watch` où les appels s'étaient terminés avant la vérification) — les deux se ressemblaient au premier abord ("ça ne marche pas comme prévu"), mais nécessitaient des solutions complètement différentes.

10. Cycle professionnel appliqué à chaque étape

À partir de ce document, et pour toute nouvelle étape technique du projet, le cycle suivant est systématiquement suivi :

1. **** Objectif**** — Qu'est-ce qu'on veut obtenir ?
 2. **** Concept**** — Comment cela fonctionne-t-il, expliqué simplement ?
 3. **** Architecture**** — Où cette brique se situe-t-elle dans le flux global ?
 4. **** Configuration**** — Quels fichiers, quelles commandes sont nécessaires ?
 5. **[FAIT] Test** — Comment vérifier concrètement que ça fonctionne ?
 6. **** Dépannage**** — Que faire précisément en cas d'échec ?
 7. **** Documentation**** — Qu'est-ce qui doit être retenu, pour l'examen ou l'entretien ?
-

1. Présentation du projet

Objectifs

Construire un IPBX (autocommutateur téléphonique privé sur IP) fonctionnel avec Asterisk, capable de : - Accueillir un appelant avec un serveur vocal interactif (IVR) utilisant la synthèse vocale Google TTS - Router l'appel vers l'un de plusieurs services (commercial, support technique, comptabilité) selon la touche pressée - Offrir des fonctionnalités d'appel avancées : mise en attente, transfert, conférence à plusieurs participants - Journaliser chaque appel (CDR — Call Detail Record) dans une base de données - Facturer les appels selon deux modes : prépayé (déduction d'un solde) et postpayé (accumulation d'une dette), avec système de codes de recharge

Architecture



Deux bases de données PostgreSQL distinctes, comme demandé dans le cahier des charges initial : -

`asterisk_cdr` : journal des appels (table `cdr`) - `a2billing` : logique de facturation (tables `comptes`, `offres`, `codes_recharge`), reliée à `asterisk_cdr` via l'extension PostgreSQL `postgres_fdw` (Foreign Data Wrapper)

Technologies utilisées

Catégorie	Technologie
Virtualisation	Oracle VirtualBox
Système d'exploitation serveur	Ubuntu Server 24.04 LTS
PBX / Central téléphonique	Asterisk 22.5.2
Synthèse vocale	Google TTS via script <code>googletts.agi</code> (Perl)
Base de données	PostgreSQL 18
Liaison inter-bases	<code>postgres_fdw</code>
Softphones de test	MicroSIP, Zoiper5, MicroSIP portable
Protocole de signalisation	SIP (via PJSIP)
Scripts serveur	Bash, PL/pgSQL

Contraintes

- Deux bases de données PostgreSQL distinctes exigées (CDR + facturation), et non une seule base avec plusieurs tables
 - Fonctionnalités de facturation devant reproduire le comportement d'A2Billing : génération de code de recharge, suppression automatique du code après usage, définition d'un coût par appel, code d'offre lié à un montant de crédit
 - Support des appels conférence à au moins 3 participants
 - Délai de rendu : 21 août ; révisions d'examen en parallèle (jusqu'au 15 août)
-

2. Prérequis

Système d'exploitation

- **Hôte** : Windows (PC personnel), avec VirtualBox installé
- **VM** : Ubuntu Server 24.04 LTS (installation minimale, sans interface graphique)

Logiciels nécessaires

Logiciel	Usage
Oracle VirtualBox (7.x)	Hyperviseur pour créer et exécuter la VM
Ubuntu Server 24.04 LTS (ISO)	Système d'exploitation du serveur Asterisk
Asterisk	Logiciel PBX central
PostgreSQL 18	Bases de données CDR et facturation
MicroSIP	Softphone (compte 6001, puis 6003 en version portable)
Zoiper5	Softphone (compte 6002)
PowerShell / Terminal	Connexion SSH depuis le PC hôte

Versions utilisées (constatées au fil du projet)

- Asterisk : 22.5.2~dfsg+~cs6.15.60671435-1
- PostgreSQL : version **18** (et non 14/15/16 comme initialement supposé — vérifié avec `pg_lsclusters`)
- Ubuntu Server : 24.04 LTS

Dépendances logicielles installées

- `asterisk-core-sounds-fr-gsm` (sons Asterisk en français)
- `perl` , `libwww-perl` , `liblwp-protocol-https-perl` (nécessaires au script `googletts.agi`)
- `sox` , `libsox-fmt-mp3` , `mpg123` (traitement audio pour la synthèse vocale)
- `postgresql` , `postgresql-contrib`
- `asterisk-modules` (contient le module `cdr_psql.so`)

Remarque : l'installation du paquet `asterisk-modules` a affiché déjà la version la plus récente , signe que le module `cdr_psql.so` était physiquement déjà présent sur le système — le problème rencontré plus tard n'était donc pas un problème d'installation manquante, mais de configuration (voir section 7).

3. Installation complète

Étape 1 — Préparation de la VM

Pourquoi Ubuntu Server et non Desktop ? Pas d'interface graphique superflue, plus léger, comportement représentatif d'un vrai serveur de production.

Pourquoi deux cartes réseau ? - Une en **NAT** : donne l'accès Internet à la VM (installation de paquets, téléchargement du script Google TTS) - Une en **Host-only** : crée une liaison directe et fiable entre le PC hôte et la VM, indépendante des aléas du Wi-Fi (voir Bug réseau, section 7)

Étapes suivies : 1. Téléchargement et installation de VirtualBox 2. Téléchargement de l'ISO Ubuntu Server 24.04 LTS 3. Création de la VM (2 Go RAM, 20 Go disque, 2 CPU si possible) 4. Installation d'Ubuntu Server avec activation d'OpenSSH dès l'installateur (pour travailler en SSH depuis le PC, plus confortable que la fenêtre VirtualBox) 5. Premier accès en SSH depuis un terminal PowerShell sur le PC hôte

Étape 2 — Résolution des problèmes réseau

(Détaillée intégralement en section 7 — Bug réseau bridge Wi-Fi, puis Bug DHCP.)

Résultat final : VM avec deux interfaces fonctionnelles : - `enp0s3` → NAT → IP dynamique type `10.0.2.x` (accès Internet) - `enp0s8` → Host-only → IP `192.168.56.101/24` (accès direct depuis le PC)

Étape 3 — Installation d'Asterisk et du module de synthèse vocale

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y asterisk asterisk-core-sounds-fr-gsm
sudo apt install -y perl libwww-perl liblwp-protocol-https-perl sox libsox-fmt-mp3 mpg123 wget
```

Récupération du script de synthèse vocale (projet open-source `zaf/asterisk-googleletts`) :

```
mkdir -p /var/lib/asterisk/agi-bin
wget -O /var/lib/asterisk/agi-bin/googleletts.agi \
  https://raw.githubusercontent.com/zaf/asterisk-googleletts/master/googleletts.agi
chmod +x /var/lib/asterisk/agi-bin/googleletts.agi
chown asterisk:asterisk /var/lib/asterisk/agi-bin/googleletts.agi
```

Attention : ce chemin (`/var/lib/asterisk/agi-bin/`) s'est révélé être le **mauvais** dossier sur cette installation — voir Bug #3 en section 7. Le script a dû être recopié dans `/usr/share/asterisk/agi-bin/` .

Vérification du service :

```
systemctl enable asterisk
systemctl restart asterisk
systemctl status asterisk --no-pager
```

Étape 4 — Configuration des comptes SIP (PJSIP)

Création du fichier `/etc/asterisk/pjsip_custom.conf` avec les comptes `6001`, `6002`, puis `6003` (voir section 5 pour le contenu complet).

Étape 5 — Écriture du dialplan (IVR + services + facturation + conférence)

Création du fichier `/etc/asterisk/extensions_custom.conf` (voir section 5 pour le contenu complet et son évolution).

Étape 6 — Installation et configuration des softphones

- **MicroSIP** (installateur classique) → compte `6001`
- **Zoiper5** → compte `6002` (nécessaire car MicroSIP ne maintient pas deux comptes enregistrés simultanément — voir Bug section 7)
- **MicroSIP portable** (version `.zip`, dans un dossier séparé) → compte `6003`, pour le test de conférence à 3 participants

Étape 7 — Installation et configuration de PostgreSQL + CDR

```
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable postgresql
```

Création de la base, de l'utilisateur et de la table CDR :

```
sudo -u postgres psql -c "CREATE USER asterisk_cdr WITH PASSWORD 'motdepassecdr';"
sudo -u postgres psql -c "CREATE DATABASE asterisk_cdr OWNER asterisk_cdr;"
sudo -u postgres psql -d asterisk_cdr -c "
CREATE TABLE cdr (
    id SERIAL PRIMARY KEY,
    calldate TIMESTAMP NOT NULL DEFAULT NOW(),
    src VARCHAR(80),
    dst VARCHAR(80),
    duration INTEGER,
    billsec INTEGER,
    disposition VARCHAR(45),
    channel VARCHAR(80)
);
"
```

Configuration du module `cdr_pgsql` (voir section 5 et Bug #7/#8 en section 7 pour les problèmes rencontrés et corrigés).

Étape 8 — Création de la base de facturation “maison” (a2billing)

Décision technique documentée en section 10. Création de la base, des tables, des fonctions PL/pgSQL, et de la liaison `postgres_fdw` avec la base CDR (voir sections 5 et 7).

Étape 9 — Automatisation de la facturation à chaque appel

Création des scripts `/usr/local/bin/facturer.sh` et `/usr/local/bin/recharger.sh`, et de la règle `sudoers` associée (voir sections 5, 7 et 12).

Étape 10 — Ajout de la fonctionnalité conférence

Ajout de l'extension `9000` avec `ConfBridge`, correction du contexte (Bug section 7), test réussi à 3 participants.

4. Toutes les commandes exécutées

Cette section liste les commandes principales dans l'ordre de leur première utilisation, avec leur rôle, ce qu'elles produisent, et les précautions associées.

#	Commande	Rôle	Ce qu'elle produit	Précaution
1	<code>ip a</code>	Affiche les interfaces réseau et leurs adresses IP	Liste des interfaces avec adresses IPv4/IPv6	Vérifier la présence d'une ligne <code>inet</code> (IPv4)
2	<code>ping -c 3 8.8.8.8</code>	Teste la connectivité IP brute (sans DNS)	Statistiques de perte de paquets	Toujours tester avec une IP avant un nom de domaine
3	<code>ping -c 3 google.com</code>	Teste connectivité + DNS ensemble	Statistiques de perte de paquets	—
4	<code>cat /etc/resolv.conf</code>	Affiche le serveur DNS utilisé	Contenu du fichier	—
5	<code>sudo apt update</code>	Rafraîchit la liste des paquets disponibles	Liste des dépôts synchronisés	Ne installe rien, juste une mise à jour de l'index
6	<code>sudo apt upgrade -y</code>	Installe les mises à jour des paquets déjà présents	Paquets mis à jour	Peut prendre plusieurs minutes
7	<code>cat /etc/netplan/*.yaml</code>	Affiche la configuration réseau persistante	Contenu YAML	Nécessite <code>sudo</code> (permission refusée sinon)
8	<code>sudo netplan apply</code>	Applique la configuration réseau	Nouvelle tentative de configuration des interfaces	Attention à la syntaxe YAML (indentation stricte)
9	<code>sudo dhclient -v enp0s3</code>	Force une requête DHCP manuelle	Logs détaillés de la négociation DHCP	Utile pour diagnostiquer un échec DHCP
10	<code>sudo apt install -y asterisk asterisk-core-sounds-fr-gsm</code>	Installe Asterisk et les sons français	Paquets installés + service créé	—
11	<code>sudo apt install -y perl libwww-perl liblwp-protocol-https-perl sox libsox-fmt-mp3 mpg123 wget</code>	Installe les dépendances de <code>googletts.agi</code>	Paquets installés	Nécessaires pour que le script AGI fonctionne
12	<code>wget -O googletts.agi https://raw.githubusercontent.com/zaf/asterisk-googletts/master/googletts.agi</code>	Télécharge le script de synthèse vocale	Fichier <code>googletts.agi</code>	Vérifier le chemin de destination (voir Bug #3)
13	<code>chmod +x googletts.agi</code>	Rend le script exécutable	Droits d'exécution ajoutés	Sans cela, Asterisk ne peut pas le lancer

#	Commande	Rôle	Ce qu'elle produit	Précaution
14	<code>chown asterisk:asterisk googletts.agi</code>	Donne la propriété du fichier à l'utilisateur système asterisk	Propriétaire changé	Nécessaire pour l'exécution par le processus Asterisk
15	<code>sudo systemctl enable asterisk</code>	Active le démarrage automatique d'Asterisk	Service activé au démarrage	—
16	<code>sudo systemctl restart asterisk</code>	(Re)démarrage Asterisk	Service relancé	—
17	<code>sudo systemctl status asterisk --no-pager</code>	Affiche l'état du service	État <code>active (running)</code> attendu	<code>--no-pager</code> évite le mode page par page
18	<code>sudo asterisk -rx "core reload"</code>	Recharge la configuration Asterisk sans redémarrer	Config relue	Plus rapide qu'un redémarrage complet
19	<code>sudo asterisk -rx "pjsip show endpoints"</code>	Liste les comptes SIP et leur état	Table des endpoints (<code>Not in use / Unavailable</code>)	—
20	<code>sudo asterisk -rvvv</code>	Ouvre la console Asterisk en mode verbeux	Logs en temps réel	Essentiel pour diagnostiquer un appel en direct
21	<code>sudo apt install -y postgresql postgresql-contrib</code>	Installe PostgreSQL	Serveur de base de données installé	—
22	<code>pg_lsclusters</code>	Liste les clusters PostgreSQL et leur statut/version	Table avec version, port, statut, chemins	Plus fiable que <code>systemctl status postgresql</code>
23	<code>sudo -u postgres psql -c "..."</code>	Exécute une commande SQL en tant qu'admin PostgreSQL	Résultat de la requête	—
24	<code>sudo -u postgres psql -d <base> -c "..."</code>	Exécute une commande SQL dans une base précise	Résultat de la requête	—
25	<code>ls -l <chemin></code>	Vérifie l'existence et les droits d'un fichier	Détails du fichier (droits, propriétaire, taille)	—
26	<code>find / -name "<motif>" 2>/dev/null</code>	Cherche un fichier sur tout le système	Chemin(s) trouvé(s)	—

#	Commande	Rôle	Ce qu'elle produit	Précaution
				2>/dev/null masque les erreurs de permission
27	<code>ldd <fichier.so> \ grep "not found"</code>	Vérifie les dépendances manquantes d'un module	Liste des bibliothèques manquantes (si any)	—
28	<code>sudo tail -30 /var/log/asterisk/messages.log</code>	Affiche les dernières lignes du log Asterisk	Contexte détaillé d'une erreur	Chercher d'abord le bon fichier avec <code>ls -la</code>
29	<code>sudo nano <fichier></code>	Édite un fichier avec l'éditeur nano	Fichier modifié après sauvegarde (<code>Ctrl+O</code> , <code>Ctrl+X</code>)	—
30	<code>sudo rm <fichier></code> puis recréation	Supprime un fichier de config corrompu/mélangé	Fichier propre recréé	Plus sûr qu'une édition manuelle complexe
31	<code>sudo chmod +x <script></code>	Rend un script exécutable	Droits d'exécution ajoutés	—
32	<code>sudo visudo -f <fichier></code>	Édite une règle sudoers en toute sécurité	Fichier de règle sudoers créé/modifié	Refuse de sauvegarder si erreur de syntaxe (silencieux)
33	<code>sudo -u asterisk sudo -u postgres <script></code>	Simule l'exécution d'un script comme le ferait Asterisk	Résultat identique à ce que produirait un vrai appel	Permet de tester sans attendre un appel réel
34	<code>SELECT * FROM <table>;</code>	Affiche le contenu d'une table	Lignes de la table	—
35	<code>\dt</code> (dans psql)	Liste les tables d'une base	Table des tables (nom, type, owner)	—
36	<code>\d <table></code> (dans psql)	Détaille la structure d'une table	Colonnes, types, index	—
37	<code>ALTER TABLE <table> OWNER TO <utilisateur>;</code>	Change le propriétaire d'une table	Propriétaire mis à jour	Nécessaire pour que d'autres utilisateurs voient les colonnes
38	<code>CREATE EXTENSION IF NOT EXISTS postgres_fdw;</code>	Active l'extension de liaison inter-bases	Extension activée	—
39	<code>CREATE SERVER ... FOREIGN DATA WRAPPER postgres_fdw ...</code>	Déclare un serveur distant	Serveur distant enregistré	—

#	Commande	Rôle	Ce qu'elle produit	Précaution
40	CREATE USER MAPPING FOR ... SERVER ...	Associe des identifiants de connexion au serveur distant	Mapping créé	—
41	CREATE FOREIGN TABLE ...	Crée une table miroir pointant vers une table distante	Table “foreign” utilisable comme une table normale (lecture)	—
42	GRANT USAGE ON FOREIGN SERVER ... TO ...	Autorise un utilisateur à utiliser la liaison distante	Droit accordé	—
43	GRANT SELECT ON <table> TO <utilisateur>;	Autorise la lecture d'une table précise	Droit accordé	Nécessaire même après le GRANT USAGE
44	SET ROLE <utilisateur>;	Change temporairement de rôle dans la session psql courante	Requêtes suivantes exécutées avec les droits de ce rôle	Utile pour tester les permissions d'un autre utilisateur

5. Configuration

5.1 /etc/netplan/00-installer-config.yaml

Rôle : configure les interfaces réseau au démarrage.

Contenu final (après correction du bug DHCP) :

```
network:
  ethernets:
    enp0s3:
      dhcp4: true
      match:
        macaddress: 08:00:27:85:e6:d9
      set-name: enp0s3
  version: 2
```

Explication ligne par ligne : - `network:` / `version: 2` : format de configuration netplan version 2 - `ethernets:` : section décrivant les cartes réseau filaires - `enp0s3:` : nom de l'interface concernée - `dhcp4: true` : **ligne critique** — demande une adresse IPv4 automatiquement en DHCP. Son absence était la cause du Bug DHCP (section 7). - `match: macaddress:` : identifie précisément l'interface par son adresse MAC (utile si les noms d'interface changent) - `set-name:` : force le nom de l'interface à `enp0s3`, indépendamment du nom que le noyau aurait pu lui attribuer

Attention : YAML est strictement sensible à l'indentation — utiliser uniquement des espaces, jamais de tabulations.

5.2 /etc/asterisk/pjsip_custom.conf

Rôle : déclare les comptes SIP utilisés par les softphones de test.

```

[transport-udp]
type=transport
protocol=udp
bind=0.0.0.0:5060

; ----- Compte 6001 -----
[6001]
type=endpoint
context=ivr-entree
disallow=all
allow=ulaw,alaw
auth=auth6001
aors=6001

[auth6001]
type=auth
auth_type=userpass
password=motdepasse6001
username=6001

[6001]
type=aor
max_contacts=1

; ----- Compte 6002 -----
[6002]
type=endpoint
context=ivr-entree
disallow=all
allow=ulaw,alaw
auth=auth6002
aors=6002

[auth6002]
type=auth
auth_type=userpass
password=motdepasse6002
username=6002

[6002]
type=aor
max_contacts=1

; ----- Compte 6003 (ajouté pour le test conférence) -----
[6003]
type=endpoint
context=ivr-entree
disallow=all
allow=ulaw,alaw
auth=auth6003
aors=6003

[auth6003]
type=auth
auth_type=userpass
password=motdepasse6003

```

```
username=6003

[6003]
type=aor
max_contacts=1
```

Explication ligne par ligne : - `[transport-udp]` : déclare le transport réseau SIP (protocole UDP, port 5060, écoute sur toutes les interfaces `0.0.0.0`) - `[6001]` `type=endpoint` : déclare un compte SIP nommé `6001`. En PJSIP, **le même nom de section peut apparaître plusieurs fois avec des `type=` différents** (`endpoint`, `aor`, `auth`) — ce n'est pas une erreur de duplication, c'est le fonctionnement normal de PJSIP - `context=ivr-entree` : détermine dans **quel contexte du dialplan** ce compte cherchera les numéros composés (voir Bug #13, section 7, sur l'importance des contextes) - `disallow=all` / `allow=ulaw,alaw` : désactive tous les codecs audio par défaut, puis n'autorise que `ulaw` et `alaw` (codecs standards, compatibles avec la majorité des softphones) - `auth=auth6001` / `aors=6001` : relie l'endpoint à sa configuration d'authentification et à son AOR (Address of Record — l'endroit où le compte peut être contacté) - `[auth6001]` : identifiants de connexion (nom d'utilisateur, mot de passe) - `[6001]` `type=aor` `max_contacts=1` : limite à une seule connexion simultanée pour ce compte

5.3 `/etc/asterisk/extensions_custom.conf` (version finale)

Rôle : le dialplan complet — IVR, routage, facturation automatique, conférence, recharge par composition.


```

[ivr-entree]
exten => 8000,1,Answer()
same => n,Wait(1)
same => n,agi(googletts.agi,"Bienvenue. Pour le service commercial, tapez 1. Pour le support technique, tapez 2. Pour la
same => n,WaitExten(5)
exten => 1,1,Goto(services,100,1)
exten => 2,1,Goto(services,101,1)
exten => 3,1,Goto(services,102,1)
exten => 9000,1,Goto(services,9000,1)
exten => _#XXX#,1,Answer()
same => n,Set(CODE=${EXTEN})
same => n,System(sudo -u postgres /usr/local/bin/recharger.sh "${CALLERID(num)}" "${CODE}" >> /var/log/asterisk/recharge
same => n,agi(googletts.agi,"Votre recharge a ete traitee. Merci d'avoir utilise notre service.",fr)
same => n,Hangup()
exten => t,1,agi(googletts.agi,"Aucune touche recue. Au revoir.",fr)
same => n,Hangup()
exten => i,1,agi(googletts.agi,"Touche invalide, veuillez recommencer.",fr)
same => n,Goto(ivr-entree,8000,2)

[facturation]
exten => h,1,NoOp(Facturation de l'appel)
same => n,System(sudo -u postgres /usr/local/bin/facturer.sh "${CALLERID(num)}" "${CDR(billsec)}" >> /var/log/asterisk/f

[services]
exten => 100,1,Answer()
same => n,Set(CHANNEL(hangup_handler_push)=facturation,h,1)
same => n,agi(googletts.agi,"Mise en relation avec le service commercial.",fr)
same => n,Dial(PJSIP/6002,20,tT)
same => n,Hangup()
exten => 101,1,Answer()
same => n,Set(CHANNEL(hangup_handler_push)=facturation,h,1)
same => n,agi(googletts.agi,"Mise en relation avec le support technique.",fr)
same => n,Dial(PJSIP/6002,20,tT)
same => n,Hangup()
exten => 102,1,Answer()
same => n,Set(CHANNEL(hangup_handler_push)=facturation,h,1)
same => n,agi(googletts.agi,"Mise en relation avec le service comptabilite.",fr)
same => n,Dial(PJSIP/6002,20,tT)
same => n,Hangup()
exten => 9000,1,Answer()
same => n,agi(googletts.agi,"Bienvenue dans la conference. Veuillez patienter, les autres participants vont vous rejoindre
same => n,ConfBridge(salle1)

```

Explication détaillée, section par section :

[ivr-entree] — point d'entrée pour tous les softphones (car `context=ivr-entree` dans

`pjsip_custom.conf`) : - `exten => 8000,1,Answer()` : décroche l'appel entrant sur le numéro 8000 - `Wait(1)` : petite pause d'une seconde avant de parler - `agi(googletts.agi, "texte", fr, 123, 1.0)` : lance le script de synthèse vocale — le `123` indique les touches qui interrompent la lecture du message (1, 2 ou 3), `1.0` est un paramètre de vitesse/volume - `WaitExten(5)` : attend jusqu'à 5 secondes qu'une touche soit pressée - `exten => 1,1,Goto(services,100,1)` : si la touche `1` est pressée, saute vers l'extension `100` du contexte `[services]` - `exten => _#XXX#,1,...` : motif de reconnaissance des codes de recharge — le `_` indique un motif (pas un numéro exact), `#` et `#` sont des caractères littéraux, chaque `X` représente un chiffre de 0 à 9. Ce motif capte

donc `#000#` à `#999#` - `exten => t,1,...` : extension spéciale `t` = déclenchée automatiquement en cas de timeout (aucune touche pressée dans le délai) - `exten => i,1,...` : extension spéciale `i` = déclenchée en cas de touche invalide

[facturation] — contexte utilisé uniquement comme “gestionnaire de raccroché” : - `exten => h,1,...` : extension spéciale `h` = exécutée automatiquement dès qu’un appel se termine (Hangup Handler), quel que soit le moyen par lequel il s’est terminé - `System(...)` : exécute une commande système externe (ici, notre script de facturation) - `${CALLERID(num)}` : variable Asterisk contenant le numéro de l’appelant - `${CDR(billsec)}` : variable contenant la durée facturable de l’appel en secondes, directement depuis le CDR de l’appel en cours

[services] — les trois services + la conférence : - `Set(CHANNEL(hangup_handler_push)=facturation,h,1)` : “branche” le gestionnaire de raccroché `[facturation]` sur cet appel — sans cette ligne, la facturation automatique ne se déclencherait jamais - `Dial(PJSIP/6002,20,tT)` : appelle le compte `6002`, avec un délai de sonnerie de 20 secondes ; l’option `tT` autorise les deux interlocuteurs à transférer l’appel en cours - `ConfBridge(salle1)` : rejoint (ou crée si elle n’existe pas) une salle de conférence nommée `salle1` — Asterisk gère automatiquement le mixage audio de tous les participants

5.4 /etc/asterisk/cdr_pgsql.conf (version finale, corrigée)

```
[global]
hostname=localhost
port=5432
dbname=asterisk_cdr
user=asterisk_cdr
password=motdepassecdr
table=cdr
```

Explication : connecte le module `cdr_pgsql` à la base `asterisk_cdr`, table `cdr`, avec les identifiants créés pour cet usage. Le choix de `hostname=localhost` reflète le fait que PostgreSQL tourne sur la même machine qu’Asterisk.

Attention : ce fichier a dû être entièrement supprimé et recréé après un bug de double section `[global]` (voir Bug #7, section 7). Toujours vérifier qu’il n’existe qu’une seule section `[global]` non commentée.

5.5 /etc/sudoers.d/asterisk-billing

```
asterisk ALL=(postgres) NOPASSWD: /usr/local/bin/facturer.sh
asterisk ALL=(postgres) NOPASSWD: /usr/local/bin/recharger.sh
```

Explication : autorise l’utilisateur système `asterisk` à exécuter **uniquement** ces deux scripts précis, en se faisant passer pour l’utilisateur `postgres`, sans qu’un mot de passe soit demandé (nécessaire puisqu’Asterisk exécute cette commande automatiquement, sans interaction humaine possible). C’est un accès volontairement restreint à deux commandes précises, pas un accès administrateur général — bonne pratique de sécurité.

Attention : ce fichier doit être créé/modifié exclusivement avec `visudo -f`, jamais avec un éditeur classique — `visudo` vérifie la syntaxe avant d'enregistrer et évite de casser tout le système `sudo` en cas d'erreur.

Cependant, ce contrôle de syntaxe est silencieux : si `visudo` rejette le fichier, il n'enregistre rien **sans message d'erreur explicite visible dans l'historique de la conversation** — d'où l'importance de toujours vérifier avec `cat` après coup (voir Bug #10, section 7).

5.6 Tables de la base `asterisk_cdr`

```
CREATE TABLE cdr (  
    id SERIAL PRIMARY KEY,  
    calldate TIMESTAMP NOT NULL DEFAULT NOW(),  
    src VARCHAR(80),  
    dst VARCHAR(80),  
    duration INTEGER,  
    billsec INTEGER,  
    disposition VARCHAR(45),  
    channel VARCHAR(80)  
);
```

Colonne	Rôle
<code>id</code>	Identifiant unique auto-incrémenté
<code>calldate</code>	Date/heure de l'appel, horodatée automatiquement
<code>src</code>	Numéro appelant (source)
<code>dst</code>	Numéro appelé (destination)
<code>duration</code>	Durée totale de l'appel en secondes (attente comprise)
<code>billsec</code>	Durée facturable (temps de conversation réel)
<code>disposition</code>	Résultat de l'appel (<code>ANSWERED</code> , <code>NO ANSWER</code> , <code>BUSY</code> ...)
<code>channel</code>	Identifiant technique du canal Asterisk utilisé

Important : cette table a dû avoir son propriétaire changé de `postgres` vers `asterisk_cdr` après création (voir Bug #8, section 7) — sans quoi le module `cdr_pgsq1` ne pouvait voir aucune colonne.

5.7 Tables et fonctions de la base a2billing

```
CREATE TABLE comptes (
  id SERIAL PRIMARY KEY,
  extension VARCHAR(20) UNIQUE NOT NULL,
  mode VARCHAR(10) NOT NULL DEFAULT 'prepaid',
  solde NUMERIC(10,2) NOT NULL DEFAULT 0,
  dette NUMERIC(10,2) NOT NULL DEFAULT 0,
  cout_par_minute NUMERIC(6,2) NOT NULL DEFAULT 5.00
);

CREATE TABLE offres (
  code_offre VARCHAR(30) PRIMARY KEY,
  montant_credit NUMERIC(10,2) NOT NULL,
  description TEXT
);

CREATE TABLE codes_recharge (
  code VARCHAR(30) PRIMARY KEY,
  montant NUMERIC(10,2) NOT NULL,
  code_offre VARCHAR(30) REFERENCES offres(code_offre)
);
```

Table	Rôle
comptes	Un enregistrement par extension SIP, avec son mode de facturation, son solde (prépayé) ou sa dette (postpayé), et son tarif
offres	Les “packs” de recharge disponibles (nom + montant de crédit)
codes_recharge	Les codes actifs, chacun lié à une offre. Un code disparaît de cette table dès qu’il est utilisé

Fonction `facturer_appel` (version finale) :

```

CREATE OR REPLACE FUNCTION facturer_appel(p_extension VARCHAR, p_billsec INTEGER)
RETURNS TEXT AS $$
DECLARE
    v_cout NUMERIC;
    v_mode VARCHAR;
BEGIN
    SELECT cout_par_minute, mode INTO v_cout, v_mode
    FROM comptes WHERE extension = p_extension;

    IF NOT FOUND THEN
        RETURN 'Compte introuvable : ' || p_extension;
    END IF;

    v_cout := ROUND((p_billsec / 60.0) * v_cout, 2);

    IF v_mode = 'prepaid' THEN
        UPDATE comptes SET solde = solde - v_cout WHERE extension = p_extension;
    ELSE
        UPDATE comptes SET dette = dette + v_cout WHERE extension = p_extension;
    END IF;

    RETURN 'Facturé ' || v_cout || ' Ar pour ' || p_extension;
END;
$$ LANGUAGE plpgsql;

```

Fonction utiliser_code_recharge (version finale, corrigée après Bug #11) :

```

CREATE OR REPLACE FUNCTION utiliser_code_recharge(p_extension VARCHAR, p_code VARCHAR)
RETURNS TEXT AS $$
DECLARE
    v_montant NUMERIC;
    v_mode VARCHAR;
BEGIN
    SELECT montant INTO v_montant FROM codes_recharge WHERE code = p_code;
    IF NOT FOUND THEN
        RETURN 'Code invalide ou deja utilise : ' || p_code;
    END IF;

    SELECT mode INTO v_mode FROM comptes WHERE extension = p_extension;
    IF NOT FOUND THEN
        RETURN 'Compte introuvable : ' || p_extension;
    END IF;

    IF v_mode = 'prepaid' THEN
        UPDATE comptes SET solde = solde + v_montant WHERE extension = p_extension;
    ELSE
        UPDATE comptes SET dette = GREATEST(dette - v_montant, 0) WHERE extension = p_extension;
    END IF;

    DELETE FROM codes_recharge WHERE code = p_code;

    RETURN 'Recharge de ' || v_montant || ' Ar effectuee pour ' || p_extension;
END;
$$ LANGUAGE plpgsql;

```

Pourquoi ces paramètres : le calcul $(\text{secondes} / 60.0) \times \text{tarif}$ convertit systématiquement en minutes avant application du tarif “par minute”. `GREATEST(dette - montant, 0)` empêche la dette de devenir négative si la recharge dépasse le montant dû. La suppression (`DELETE`) du code après usage, plutôt qu’un simple marquage “utilisé”, correspond explicitement à la demande initiale (“tout code de recharge qui est déjà utilisé est supprimé automatiquement de la BD”).

5.8 Liaison postgres_fdw (base a2billing → base asterisk_cdr)

```
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER serveur_cdr
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'localhost', dbname 'asterisk_cdr', port '5432');

CREATE USER MAPPING FOR a2billing_user
SERVER serveur_cdr
OPTIONS (user 'asterisk_cdr', password 'motdepassecdr');

CREATE FOREIGN TABLE cdr_distant (
    id INTEGER,
    calldate TIMESTAMP,
    src VARCHAR(80),
    dst VARCHAR(80),
    duration INTEGER,
    billsec INTEGER,
    disposition VARCHAR(45),
    channel VARCHAR(80)
) SERVER serveur_cdr OPTIONS (schema_name 'public', table_name 'cdr');

GRANT USAGE ON FOREIGN SERVER serveur_cdr TO a2billing_user;
GRANT SELECT ON cdr_distant TO a2billing_user;
```

Pourquoi ce choix : le cahier des charges exige deux bases PostgreSQL séparées (pas une seule base avec deux tables). `postgres_fdw` est l'outil PostgreSQL standard pour permettre à une base de lire les données d'une autre base sans les fusionner ni les dupliquer.

5.9 /etc/asterisk/queues.conf

Rôle : définit la file d'attente utilisée pour le repli automatique vers un second agent si le premier est occupé.

```
[service_client]
strategy = ringall
timeout = 20
ringinuse = no
member => PJSIP/6002
member => PJSIP/6003

; Alternative documentée mais NON appliquée (config active = ringall) :
; strategy = leastrecent
; → fait sonner en priorité l'agent resté libre le plus longtemps
; → répartition plus équitable entre agents, au prix d'une sonnerie
; un peu moins immédiate qu'avec ringall (un seul agent sonne à
; la fois plutôt que tous les agents libres en même temps)
```

Explication ligne par ligne : - `strategy = ringall` : stratégie retenue — fait sonner **tous** les agents libres simultanément, le premier à décrocher prend l'appel - `timeout = 20` : durée maximale d'attente en file avant abandon de l'appel si aucun agent ne se libère - `ringinuse = no` : empêche un agent déjà en communication de

recevoir un second appel simultané — condition nécessaire au bon fonctionnement du repli - `member => PJSIP/6002 / PJSIP/6003` : les deux agents disponibles pour cette file

Remarque : le choix de `ringall` plutôt que `leastrecent` est documenté comme décision assumée en section 16.5 — aucune exigence du cahier des charges ne demandait une répartition équitable entre agents, et `ringall` était déjà testé et validé.

6. Chronologie complète du projet

1. Analyse des notes de cours (photos manuscrites) et définition du plan de projet en 4 grandes étapes (IVR/Asterisk, CDR, A2Billing/facturation, conférence)
2. Préparation de l'environnement : VirtualBox, téléchargement Ubuntu Server, création de la VM
3. **Bug réseau n°1** : mode Bridged en échec sur Wi-Fi → bascule vers architecture NAT + Host-only
4. Installation d'Ubuntu Server, premier accès SSH
5. **Bug réseau n°2** : pas d'IPv4 sur l'interface NAT → correction du fichier netplan (`dhcp4: true`)
6. Installation d'Asterisk et des dépendances de synthèse vocale
7. **Bug n°3** : chemin du script `googletts.agi` incorrect → recopie au bon endroit
8. Configuration des comptes SIP `6001` et `6002` (PJSIP)
9. Écriture du dialplan IVR initial (menu, routage vers 3 services)
10. Installation et configuration de MicroSIP (compte 6001)
11. **Bug n°4** : transfert vers 6002 échoue (compte non enregistré)
12. **Bug n°5** : impossible de garder 6001 et 6002 enregistrés simultanément dans une seule fenêtre MicroSIP → installation de Zoiper pour 6002
13. Test complet réussi de l'IVR avec transfert vers un agent
14. Test et validation de la mise en attente (Hold)
15. Test des 3 services (commercial, support, comptabilité) — tous validés
16. Installation de PostgreSQL
17. **Bug n°6** : confusion sur le statut `active (exited)` de `postgresql.service` → clarification avec `pg_lsclusters`
18. Création de la base et de la table CDR
19. **Bug n°7** : module `cdr_pgsqll.so` refuse de se charger (double section `[global]`) → fichier de config recréé proprement
20. **Bug n°8** : erreur de syntaxe SQL à l'insertion (`syntax error at or near ")"`) → correction du propriétaire de la table (`ALTER TABLE ... OWNER TO`)
21. Test réussi : premier appel réellement enregistré dans le CDR
22. Recherche et décision technique : abandon de l'installation du logiciel A2Billing officiel (incompatible PostgreSQL) au profit d'une logique de facturation "maison" en PL/pgSQL
23. Création de la base `a2billing` , des tables `comptes` , `offres` , `codes_recharge`
24. Insertion de données de test (comptes 6001 prépayé, 6002 postpayé, offres, codes)
25. Mise en place de la liaison `postgres_fdw` entre `a2billing` et `asteriskcdr`
26. **Bug n°9** : erreurs de permissions sur la liaison `postgres_fdw` → corrections successives (`SET ROLE` , `GRANT SELECT`)
27. Création et test manuel de la fonction `facturer_appel`
28. **Bug n°10** : facturation automatique après appel en échec (`sudo: I'm sorry asterisk...`) → diagnostic en plusieurs étapes (règle sudoers jamais enregistrée, puis script inexistant) → création complète et vérifiée du script `facturer.sh` + règle sudoers

29. Test réussi : facturation automatique confirmée après un appel réel
 30. Création de la fonction `utiliser_code_recharge` , test manuel réussi
 31. Discussion sur le format des codes de recharge → adoption du format `#XXX#` (inspiré des codes USSD opérateur)
 32. Ajout de la reconnaissance de motif `_#XXX#` dans le dialplan pour permettre la recharge par composition téléphonique directe
 33. **Bug n°11** : recharge d'un compte postpayé créait un solde au lieu de rembourser la dette → correction de la fonction `utiliser_code_recharge`
 34. **Bug n°12** : recharge par téléphone échoue silencieusement (commande tronquée par des caractères spéciaux `, et ;`) → solution finale avec script dédié `recharger.sh`
 35. Test réussi : recharge complète par composition téléphonique validée
 36. Ajout de la fonctionnalité conférence (`ConfBridge` , extension 9000)
 37. **Bug n°13** : extension 9000 introuvable (mauvais contexte) → ajout d'une passerelle dans `[ivr-entree]`
 38. Test réussi à 2 participants, puis blocage sur l'ajout d'un 3^e participant (limitation logicielle des softphones)
 39. **Bug n°14** : Linphone se ferme au démarrage → abandon, remplacement par une copie portable de MicroSIP
 40. Test final réussi : conférence à 3 participants simultanés (6001, 6002, 6003)
 41. Tests manuels complémentaires : facturation postpayée, génération/utilisation de codes de recharge supplémentaires
 42. Rédaction de la documentation technique (ce document)
-

7. Toutes les difficultés rencontrées

Difficulté n°1 — Mode réseau Bridged inutilisable sur Wi-Fi

- **Contexte** : premières tentatives d'accès Internet depuis la VM, réseau configuré en "Bridged Adapter" sur la carte Wi-Fi de l'hôte
- **Message d'erreur exact** : `Pping unreachable / Destination host unreachable` (via `ping 8.8.8.8`)
- **Analyse** : élimination immédiate du DNS comme cause, puisque le test utilisait une IP brute et non un nom de domaine
- **Cause** : limitation connue de VirtualBox — le mode Bridged sur une carte Wi-Fi échoue souvent car le protocole 802.11 n'autorise généralement qu'une seule adresse MAC associée par carte physique, rejetant les trames de la VM
- **Solution appliquée** : passage à une architecture à deux adaptateurs (NAT pour Internet + Host-only pour la liaison PC ↔ VM)
- **Pourquoi cette solution fonctionne** : le mode NAT ne nécessite pas de manipulation d'adresses MAC visibles sur le réseau physique, et le mode Host-only crée un réseau privé virtuel géré entièrement par VirtualBox, indépendant du matériel Wi-Fi

Difficulté n°2 — Absence d'adresse IPv4 sur l'interface NAT

- **Contexte** : après passage en NAT, `sudo apt update` échouait systématiquement
- **Message d'erreur exact** : Erreur temporaire de résolution de « `archive.ubuntu.com` », Impossible de récupérer `http://archive.ubuntu.com/...`
- **Analyse** : `ip a` a révélé l'absence totale de ligne `inet` (IPv4) sur `enp0s3`, uniquement des adresses IPv6
- **Cause** : absence de la ligne `dhcp4: true` dans le fichier `netplan` — l'interface ne demandait jamais d'adresse IP
- **Solution appliquée** : ajout de `dhcp4: true` dans `/etc/netplan/00-installer-config.yaml`, puis `sudo netplan apply`
- **Pourquoi cette solution fonctionne** : cette ligne active le client DHCP sur l'interface, qui peut alors obtenir une adresse auprès du serveur DHCP fourni automatiquement par VirtualBox en mode NAT

Difficulté n°3 — Script `googletts.agi` introuvable

- **Contexte** : premier test d'appel vers l'IVR (numéro 8000)
- **Message d'erreur exact** : `WARNING: res_agi.c:2386 launch_script: Failed to execute '/usr/share/asterisk/agi-bin/googletts.agi': File does not exist.`
- **Analyse** : observation en direct via `asterisk -rvvv` pendant l'appel
- **Cause** : le script avait été placé dans `/var/lib/asterisk/agi-bin/`, alors qu'Asterisk cherchait dans `/usr/share/asterisk/agi-bin/` sur cette installation précise
- **Solution appliquée** : copie du script vers le bon dossier, avec les droits d'exécution et le bon propriétaire

- **Pourquoi cette solution fonctionne** : Asterisk trouve désormais le script à l'endroit exact où il le cherche, confirmé par le message `Launched AGI Script` dans les logs suivants

[EN COURS] Méthode retenue pour éviter ce bug lors d'une réinstallation future : ne jamais supposer le chemin AGI par convention. Avant de télécharger le script, demander directement à Asterisk où il regarde :

```
grep "astagidir" /etc/asterisk/asterisk.conf
```

Cette ligne (section `[directories]` du fichier de configuration central d'Asterisk) donne le chemin exact et fiable, quelle que soit la distribution Linux utilisée — plus besoin d'attendre une erreur pour le découvrir après coup.

Difficulté n°4 — Transfert d'appel vers 6002 en échec

- **Contexte** : test du routage IVR → service → transfert
- **Message d'erreur exact** : `ERROR: res_pjsip.c:993 ast_sip_create_dialog_uac: Endpoint '6002': Could not create dialog to invalid URI '6002'. Is endpoint registered and reachable?`
- **Analyse** : le message indique littéralement que l'endpoint n'est ni enregistré ni joignable
- **Cause** : aucun softphone n'était encore connecté avec le compte `6002`
- **Solution appliquée** : installation et configuration d'un second softphone pour ce compte
- **Pourquoi cette solution fonctionne** : une fois enregistré, l'endpoint devient joignable et le `Dial()` peut aboutir

Difficulté n°5 — Impossible de garder deux comptes SIP actifs simultanément dans MicroSIP

- **Contexte** : ajout du compte `6002` comme second compte dans la même fenêtre MicroSIP
- **Message d'erreur exact (indirect, observé dans les logs Asterisk)** : alternance de messages `Endpoint 6001 is now Unreachable` / `Endpoint 6002 is now Reachable`, l'un au détriment de l'autre
- **Analyse** : le compte sélectionné (coché) dans le menu MicroSIP semblait être le seul réellement maintenu enregistré
- **Cause** : MicroSIP, dans ce mode d'utilisation à comptes multiples dans une seule fenêtre, ne maintient qu'un seul enregistrement actif à la fois
- **Solution appliquée** : utilisation de deux applications softphone distinctes et indépendantes (MicroSIP pour `6001`, Zoiper pour `6002`)
- **Pourquoi cette solution fonctionne** : deux processus applicatifs séparés maintiennent chacun leur propre enregistrement SIP indépendamment, confirmé par `pjsip show endpoints` affichant les deux comptes `Not in use` simultanément

Difficulté n°6 — Confusion sur le statut du service PostgreSQL

- **Contexte** : vérification du service après installation
- **Message affiché** : `Active: active (exited)`

- **Analyse** : ce statut a été initialement interprété à tort comme un problème
- **Cause réelle** : sur Ubuntu/Debian, `postgresql.service` est un service “chef d’orchestre” qui démarre le vrai service versionné puis se termine lui-même — `exited` signifie “tâche de démarrage terminée avec succès”, pas “en panne”
- **Solution appliquée** : utilisation de `pg_lsclusters` pour obtenir le statut réel (`online`) et la version exacte (18)
- **Pourquoi cette solution fonctionne** : `pg_lsclusters` interroge directement l’état des clusters PostgreSQL, indépendamment du nom du service `systemd`

Difficulté n°7 — Module `cdr_pgsql.so` refusant de se charger

- **Contexte** : première tentative de chargement du module après configuration de `cdr_pgsql.conf`
- **Message d’erreur exact** : `Unable to load module cdr_pgsql.so` puis, dans les logs : `cdr_pgsql.c: cdr_pgsql configuration contains no global section, skipping module load.`
- **Analyse** : vérification des dépendances système (`ldd`) → aucune dépendance manquante ; donc le problème venait du fichier de configuration lui-même
- **Cause** : le fichier contenait deux sections `[global]` (une dans l’exemple commenté d’origine du fichier, une ajoutée avec les vraies valeurs), ce qui perturbait le parseur du module
- **Solution appliquée** : suppression complète du fichier et recréation propre avec une seule section `[global]`
- **Pourquoi cette solution fonctionne** : confirmé par le message `Loaded cdr_pgsql.so` après la correction

Difficulté n°8 — Erreur SQL à l’insertion des CDR

- **Contexte** : après chargement réussi du module, la table restait vide malgré des appels réels
- **Message d’erreur exact** :

```
ERROR: cdr_pgsql.c: HARD ERROR! Attempted reconnection failed. DROPPING CALL RECORD! /
Reason: ERROR: syntax error at or near ")"
```
- **Analyse** : `\dt` a révélé que la table `cdr` appartenait à l’utilisateur `postgres`, pas à `asterisk_cdr`
- **Cause** : en se connectant avec l’utilisateur `asterisk_cdr` (propriétaire de la base mais pas de cette table précise), le module ne voyait aucune colonne, générant une requête `INSERT INTO cdr () VALUES ()` invalide
- **Solution appliquée** : `ALTER TABLE cdr OWNER TO asterisk_cdr;`
- **Pourquoi cette solution fonctionne** : confirmé par l’apparition d’une vraie ligne dans `SELECT * FROM cdr;` après un appel test

Difficulté n°9 — Erreurs de permission sur la liaison `postgres_fdw`

- **Contexte** : première tentative de lecture de la table distante `cdr_distant` depuis la base `a2billing`
- **Messages d’erreur exacts** : `ERROR: user mapping not found for user "postgres", server "serveur_cdr"` puis `ERROR: permission denied for foreign table cdr_distant`
- **Analyse** : la première erreur venait d’un test effectué avec le mauvais utilisateur (`postgres` au lieu de `a2billing_user`) ; la seconde d’un droit manquant
- **Cause** : le `USER MAPPING` avait été créé uniquement pour `a2billing_user`, et le `GRANT USAGE` sur le serveur ne suffisait pas — il fallait aussi un `GRANT SELECT` explicite sur la table

- **Solution appliquée** : `SET ROLE a2billing_user;` pour tester avec le bon utilisateur, puis `GRANT SELECT ON cdr_distant TO a2billing_user;`
- **Pourquoi cette solution fonctionne** : confirmé par l’affichage correct des 3 lignes de CDR depuis la base `a2billing`

Difficulté n°10 — Facturation automatique non déclenchée après un appel

- **Contexte** : test du hangup handler après ajout de la ligne `System(...)` dans le dialplan
- **Message d’erreur exact** : `sudo: I'm sorry asterisk. I'm afraid I can't do that`
- **Analyse** : message caractéristique d’un refus de permission `sudo .` Test isolé avec `sudo -u asterisk sudo -u postgres <commande>` pour reproduire sans attendre un vrai appel
- **Cause (en deux temps)** :
 1. Le fichier `/etc/sudoers.d/asterisk-billing` n’existait pas du tout (la première tentative de création via `visudo` n’avait jamais été effectivement enregistrée)
 2. Après recréation de la règle, nouvelle erreur `command not found` : le script `/usr/local/bin/facturer.sh` lui-même n’avait pas encore été créé
- **Solution appliquée** : création réelle et vérifiée du script, recréation de la règle sudoers avec vérification immédiate par `cat`
- **Pourquoi cette solution fonctionne** : confirmé par un test manuel réussi (Facturé 1.25 Ar pour 6001), puis par un test en conditions réelles montrant le solde diminuer automatiquement après un appel, sans intervention manuelle

Difficulté n°11 — Logique de recharge incohérente pour les comptes postpayés

- **Contexte** : test de la fonction `utiliser_code_recharge` sur le compte postpayé 6002
- **Symptôme (pas un message d’erreur, mais un résultat incohérent)** : le compte affichait à la fois un solde positif ET une dette , alors qu’un compte postpayé ne devrait jamais avoir de solde
- **Analyse** : relecture du code de la fonction `utiliser_code_recharge`
- **Cause** : la fonction ajoutait systématiquement au solde , sans distinguer le mode (prépayé/postpayé) du compte concerné
- **Solution appliquée** : réécriture de la fonction pour vérifier le mode du compte et appliquer soit `solde += montant` (prépayé), soit `dette = GREATEST(dette - montant, 0)` (postpayé)
- **Pourquoi cette solution fonctionne** : confirmé par un test où la dette d’un compte passe correctement de 30 à 10 après une recharge de 20, sans création de solde parasite

Difficulté n°12 — Recharge par composition téléphonique échouant silencieusement

- **Contexte** : premier test de composition du code `#222#` depuis un softphone
- **Symptôme** : le message vocal de confirmation se jouait normalement, mais aucune donnée n’était modifiée en base, et le fichier log n’était même pas créé
- **Analyse** : observation en direct dans la console Asterisk a révélé que la commande `System(...)` était tronquée à l’affichage :

```
System(..., "SELECT utiliser_code_recharge('6002', '#222#') in new stack
```

- **Cause** : deux caractères ont une signification spéciale dans le dialplan Asterisk, même à l'intérieur de guillemets : la virgule `,` (séparateur d'arguments) et le point-virgule `;` (début de commentaire)
- **Solution appliquée (en deux temps)** :
 1. Tentative d'échappement des caractères spéciaux (`\,` et `\;`) — insuffisante seule, car elle a révélé un second problème identique à la Difficulté n°10 (commande non couverte par la règle sudoers)
 2. Solution finale : création d'un script dédié `/usr/local/bin/recharger.sh`, ajouté à la règle sudoers, appelé depuis le dialplan sans caractères spéciaux à gérer
- **Pourquoi cette solution fonctionne** : en déplaçant la logique SQL complexe dans un script shell séparé, le dialplan n'a plus qu'à transmettre deux arguments simples, sans virgule ni point-virgule à interpréter

Difficulté n°13 — Extension conférence (9000) introuvable

- **Contexte** : premier test d'appel vers le numéro de conférence
- **Message d'erreur exact** : L'appel a échoué : 9000 – Numérotation incorrecte ou numéro inexistant (affiché côté MicroSIP)
- **Analyse** : vérification du fichier de dialplan — l'extension existait bel et bien, mais dans le mauvais contexte
- **Cause** : les softphones sont configurés avec `context=ivr-entree`, mais l'extension 9000 avait été ajoutée uniquement dans le contexte `[services]`, invisible pour un softphone qui compose directement
- **Solution appliquée** : ajout d'une ligne passerelle dans `[ivr-entree]` : `exten => 9000,1,Goto(services,9000,1)`
- **Pourquoi cette solution fonctionne** : confirmé par le message vocal d'accueil de la conférence qui se joue correctement lors du test suivant

Difficulté n°14 — Application Linphone se fermant automatiquement

- **Contexte** : tentative d'installation d'un troisième softphone pour tester la conférence à 3 participants
- **Symptôme** : l'application se ferme immédiatement après le démarrage, sans message d'erreur exploitable
- **Analyse** : recherche rapide confirmant un bug connu, non trivial à diagnostiquer sur cette configuration précise
- **Cause** : non déterminée précisément (bug de compatibilité potentiel, non documenté dans l'historique de la conversation)
- **Solution appliquée** : abandon de Linphone, remplacement par une copie portable de MicroSIP dans un dossier séparé (processus indépendant, contournant le problème de MicroSIP à comptes multiples vu en Difficulté n°5)
- **Pourquoi cette solution fonctionne** : chaque copie portable de MicroSIP est un processus Windows indépendant avec son propre fichier de configuration, permettant un enregistrement SIP simultané avec les autres softphones

8. Toutes les erreurs rencontrées

Historique complet, y compris les erreurs mineures ou de manipulation, dans l'ordre chronologique.

#	Erreur / message	Contexte	Gravité
1	Erreur temporaire de résolution de « archive.ubuntu.com »	Premier apt update	Bloquante
2	Pping unreachable / Destination host unreachable	Test réseau bridge Wi-Fi	Bloquante
3	cat: /etc/netplan/*.yaml: Permission denied	Lecture du fichier netplan sans sudo	Mineure (manipulation)
4	Timeout de connexion SSH	Tentative de connexion après changement de config réseau	Bloquante temporairement (résolue en vérifiant l'IP et la carte host-only du PC)
5	WARNING: Failed to execute '/usr/share/asterisk/agi-bin/googletts.agi': File does not exist.	Premier test d'appel IVR	Bloquante
6	ERROR: Could not create dialog to invalid URI '6002'	Transfert vers un compte non enregistré	Bloquante
7	Appel initié depuis le mauvais compte (6002 au lieu de 6001) dans MicroSIP	Confusion d'interface lors du test avec plusieurs comptes	Mineure (erreur de manipulation)
8	Cercle de statut MicroSIP resté gris	Compte SIP non enregistré	Bloquante temporairement
9	Active: active (exited) mal interprété comme une panne	Vérification du service PostgreSQL	Mineure (fausse alerte)
10	Unit postgresql@16-main.service not found	Tentative de vérification avec le mauvais numéro de version	Mineure (hypothèse incorrecte corrigée)
11	Unable to load module cdr_pgsql.so	Chargement initial du module CDR	Bloquante
12	cdr_pgsql configuration contains no global section, skipping module load.	Cause précise du blocage précédent	Bloquante
13	ERROR: syntax error at or near ")"	Insertion des CDR après chargement du module	Bloquante

#	Erreur / message	Contexte	Gravité
14	ls: impossible d'accéder à '/usr/lib/ asterisk/modules/' : Aucun fichier ou répertoire de ce type	Recherche du dossier de modules au mauvais endroit	Mineure (chemin incorrect, corrigé avec find)
15	tail: impossible d'ouvrir '/var/log/ asterisk/full' en lecture : Aucun fichier ou répertoire de ce type	Recherche du fichier de log au mauvais nom	Mineure (chemin incorrect, corrigé avec ls -la)
16	ERROR: user mapping not found for user "postgres", server "serveur_cdr"	Test de postgres_fdw avec le mauvais utilisateur	Mineure (erreur de manipulation)
17	ERROR: permission denied for foreign table cdr_distant	Droit GRANT SELECT manquant	Bloquante
18	sudo: I'm sorry asterisk. I'm afraid I can't do that	Facturation automatique, permission sudoers	Bloquante (répétée 2 fois pour 2 causes différentes)
19	cat: /etc/sudoers.d/ asterisk-billing: No such file or directory	Vérification après une création de règle sudoers non enregistrée	Bloquante
20	sudo: '/usr/local/bin/ facturer.sh': command not found	Script pas encore créé au moment du test	Bloquante
21	ls: impossible d'accéder à '/usr/ local/bin/ facturer.sh' : Aucun fichier ou répertoire de ce type	Confirmation de l'erreur précédente	Bloquante
22	Solde et dette incohérents simultanément sur un compte postpayé	Bug logique de la fonction de recharge (pas un message d'erreur système)	Bloquante (logique métier)
23	Commande system() tronquée dans les logs Asterisk	Caractères spéciaux non échappés dans le dialplan	Bloquante
24	sudo: '/usr/local/bin/ recharger.sh': command	Avant création du script de recharge	Bloquante

#	Erreur / message	Contexte	Gravité
	not found (implicite, même famille que #20)		
25	L'appel a échoué : 9000 – Numérotation incorrecte ou numéro inexistant	Extension conférence dans le mauvais contexte	Bloquante
26	Linphone se ferme automatiquement après démarrage	Tentative d'installation d'un 3 ^e softphone	Bloquante (contournée)
27	Fichier extensions_custom.conf mélangé (ancienne + nouvelle version après collage partiel)	Erreur de manipulation lors d'une édition dans nano	Mineure (corrigée par suppression/recréation)

9. Toutes les corrections apportées

Correction	Fichier / élément concerné	Raison
Ajout de <code>dhcp4: true</code>	<code>/etc/netplan/00-installer-config.yaml</code>	Activer la demande automatique d'adresse IPv4
Passage de Bridged à NAT + Host-only	Paramètres réseau VirtualBox de la VM	Contourner la limitation du bridging sur Wi-Fi
Déplacement du script <code>googletts.agi</code>	de <code>/var/lib/asterisk/agi-bin/</code> vers <code>/usr/share/asterisk/agi-bin/</code>	Placer le script dans le dossier réellement utilisé par Asterisk
Suppression et recréation de <code>cdr_pgsql.conf</code>	<code>/etc/asterisk/cdr_pgsql.conf</code>	Éliminer la double section <code>[global]</code>
Changement de propriétaire de la table <code>cdr</code>	Base <code>asterisk_cdr</code>	Permettre à l'utilisateur <code>asterisk_cdr</code> de voir les colonnes de sa propre table
Ajout de <code>GRANT SELECT ON cdr_distant</code>	Base <code>a2billing</code>	Autoriser la lecture effective de la table distante, en plus du <code>GRANT USAGE</code> sur le serveur
Recréation complète du script <code>facturer.sh</code> et de la règle <code>sudoers</code>	<code>/usr/local/bin/facturer.sh</code> , <code>/etc/sudoers.d/asterisk-billing</code>	La première tentative de création n'avait pas été effectivement enregistrée
Réécriture de <code>utiliser_code_recharge</code> pour distinguer prépayé/postpayé	Base <code>a2billing</code> , fonction PL/pgSQL	Éviter la création d'un solde incohérent sur un compte postpayé
Remplacement de la commande SQL directe par un script dédié <code>recharger.sh</code>	<code>/etc/asterisk/extensions_custom.conf</code> , <code>/usr/local/bin/recharger.sh</code>	Éviter les problèmes de caractères spéciaux (, et ;) non échappables de façon fiable dans le dialplan
Ajout d'une extension passerelle <code>exten => 9000,1,Goto(services,9000,1)</code>	<code>[ivr-entree]</code> dans <code>extensions_custom.conf</code>	Rendre l'extension conférence joignable depuis le contexte utilisé par les softphones
Remplacement de Linphone par une copie portable de MicroSIP	Softphone du compte 6003	Contourner le plantage au démarrage de Linphone
Suppression et recréation complète de <code>extensions_custom.conf</code>	<code>/etc/asterisk/extensions_custom.conf</code>	Corriger un mélange accidentel entre deux versions du fichier lors d'une édition manuelle

10. Décisions techniques

Décision 1 — Architecture réseau NAT + Host-only plutôt que Bridged

Contexte : échec du mode Bridged sur carte Wi-Fi. **Choix** : deux adaptateurs réseau (NAT pour Internet, Host-only pour la liaison PC ↔ VM), plutôt que d'insister sur le mode Bridged ou de tenter un partage de connexion complexe.

Justification : solution plus fiable et adaptée au cas d'usage précis (softphones sur le même PC que l'hyperviseur), sans dépendre du comportement du matériel Wi-Fi.

Décision 2 — Deux softphones distincts plutôt qu'un seul avec plusieurs comptes

Contexte : MicroSIP ne maintenait pas deux comptes enregistrés simultanément dans une seule fenêtre. **Choix** : utilisation de deux applications différentes (MicroSIP, Zoiper), puis d'une troisième instance via une copie portable, plutôt que de chercher un réglage caché dans MicroSIP. **Justification** : solution plus rapide et plus fiable, chaque application étant un processus indépendant garantissant un enregistrement SIP permanent.

Décision 3 — Abandon du logiciel A2Billing officiel au profit d'une logique "maison"

Contexte : le cahier des charges demande explicitement deux bases **PostgreSQL** distinctes. Recherche effectuée : le logiciel A2Billing officiel (dépôt Star2Billing) ne fournit et ne maintient qu'un schéma de base de données

MySQL ; aucun schéma PostgreSQL n'est distribué ni maintenu dans les versions actuelles, malgré une ancienne mention "(Postgresql)" dans la documentation générale du produit. **Choix** : reconstruire nous-mêmes, en PostgreSQL, la logique fonctionnelle décrite par les notes de cours (comptes, offres, codes de recharge, facturation prépayée/postpayée), plutôt que d'installer le paquet logiciel A2Billing en MySQL (ce qui contredirait la consigne des deux bases PostgreSQL) ou de tenter un portage manuel complet du schéma A2Billing vers PostgreSQL (jugé disproportionné en temps pour la portée du projet). **Justification** : cohérence totale avec la consigne explicite ("2 BD dans PostgreSQL"), maîtrise complète du code produit (utile pour la soutenance orale), et fidélité aux fonctionnalités précises listées dans les notes de cours plutôt qu'à un produit logiciel particulier.

Décision 4 — Liaison entre les deux bases via `postgres_fdw` plutôt que fusion en une seule base

Contexte : besoin de faire correspondre les appels du CDR avec les comptes de facturation, tout en respectant l'exigence de deux bases séparées. **Choix** : utilisation de l'extension standard PostgreSQL `postgres_fdw` pour créer une table miroir en lecture seule, plutôt que de dupliquer les données ou de fusionner les deux bases en une seule. **Justification** : solution reconnue et documentée de PostgreSQL pour ce cas d'usage exact (accès inter-bases), qui respecte l'architecture à deux bases distinctes demandée.

Décision 5 — Scripts shell dédiés plutôt que commandes SQL directes dans le dialplan

Contexte : échecs répétés liés à l'interprétation de caractères spéciaux (, et ;) par le parseur du dialplan Asterisk lors de l'exécution de commandes SQL complexes via `system()`. **Choix** : création de scripts shell dédiés (`facturer.sh` , `recharger.sh`) encapsulant la logique SQL, appelés depuis le dialplan avec de simples arguments. **Justification** : approche plus robuste et plus lisible que la multiplication de caractères d'échappement

dans le dialplan ; limite aussi la surface d'exposition de la commande `sudo` à des scripts précis et contrôlés (voir configuration `sudoers`).

Décision 6 — Format de code de recharge `#xxx#` plutôt qu'un format alphanumérique

Contexte : besoin d'un format de code facile à retenir et à composer directement au clavier téléphonique. **Choix** : adoption d'un format inspiré des codes USSD des opérateurs mobiles (type `#111#`), plutôt qu'un format alphanumérique complexe (type `REC-AAA111`). **Justification** : format nativement composable au clavier d'un téléphone (chiffres uniquement), cohérent avec un usage réel de recharge téléphonique.

11. Arborescence finale

D'après les fichiers explicitement créés ou modifiés au cours de la conversation.

```
/etc/asterisk/
├─ pjsip_custom.conf          (comptes SIP 6001, 6002, 6003)
├─ extensions_custom.conf     (dialplan complet : IVR, services, facturation, conférence)
├─ cdr_pgsql.conf             (connexion module CDR ↔ PostgreSQL)
└─ rtp_custom.conf            (plage de ports RTP – préparé en tout début de projet)

/etc/netplan/
└─ 00-installer-config.yaml   (configuration réseau, avec dhcp4: true)

/etc/sudoers.d/
└─ asterisk-billing           (autorisations sudo pour les scripts de facturation)

/usr/local/bin/
├─ facturer.sh               (script de facturation automatique)
└─ recharger.sh               (script d'utilisation d'un code de recharge)

/usr/share/asterisk/agi-bin/
└─ googletts.agi              (script de synthèse vocale Google TTS)

/var/log/asterisk/
├─ messages.log               (log principal Asterisk)
├─ facturation.log             (log dédié à la facturation automatique)
└─ recharge.log                (log dédié aux recharges)

Bases de données PostgreSQL :
├─ asterisk_cdr
│   └─ table: cdr
└─ a2billing
    ├── table: comptes
    ├── table: offres
    ├── table: codes_recharge
    ├── table étrangère: cdr_distant (via postgres_fdw → asterisk_cdr.cdr)
    ├── fonction: facturer_appel(extension, billsec)
    └─ fonction: utiliser_code_recharge(extension, code)
```

Remarque : l'arborescence des fichiers créés localement sur le PC de travail avant transfert (/mnt/user-data/outputs/voip-project/... , contenant les premiers brouillons de scripts et de configuration produits en tout début de projet) n'a pas été utilisée en pratique — l'ensemble du travail a finalement été réalisé directement sur la VM via `nano` , comme le montre l'historique des commandes.

12. Scripts

12.1 /usr/local/bin/facturer.sh

```
#!/bin/bash
EXT="$1"
SEC="$2"
psql -d a2billing -c "SELECT facturer_appel('${EXT}', ${SEC});"
```

Commentaire ligne par ligne : - `#!/bin/bash` : indique au système que ce script doit être interprété par Bash - `EXT="$1"` : récupère le premier argument passé au script (le numéro d'extension de l'appelant) et le stocke dans la variable `EXT` - `SEC="$2"` : récupère le second argument (la durée facturable en secondes) et le stocke dans `SEC` - `psql -d a2billing -c "..."` : se connecte à la base `a2billing` et exécute la commande SQL qui appelle la fonction `facturer_appel` avec les deux valeurs reçues

Appelé depuis le dialplan par :

```
System(sudo -u postgres /usr/local/bin/facturer.sh "${CALLERID(num)}" "${CDR(billsec)}" >> /var/log/asterisk/facturation.
```

12.2 /usr/local/bin/recharger.sh

```
#!/bin/bash
EXT="$1"
CODE="$2"
psql -d a2billing -c "SELECT utiliser_code_recharge('${EXT}', '${CODE}');"
```

Commentaire ligne par ligne : - `#!/bin/bash` : interpréteur Bash - `EXT="$1"` : premier argument — le numéro de l'extension qui recharge son compte - `CODE="$2"` : second argument — le code de recharge composé par l'utilisateur - `psql -d a2billing -c "..."` : exécute la fonction `utiliser_code_recharge` avec ces deux valeurs

Appelé depuis le dialplan par :

```
System(sudo -u postgres /usr/local/bin/recharger.sh "${CALLERID(num)}" "${CODE}" >> /var/log/asterisk/recharge.log 2>&1)
```

12.3 /etc/sudoers.d/asterisk-billing

```
asterisk ALL=(postgres) NOPASSWD: /usr/local/bin/facturer.sh
asterisk ALL=(postgres) NOPASSWD: /usr/local/bin/recharger.sh
```

Commentaire : - `asterisk` : l'utilisateur système autorisé (celui sous lequel tourne le processus Asterisk) - `ALL=(postgres)` : autorisé sur toutes les machines (`ALL`), en se faisant passer pour l'utilisateur `postgres` - `NOPASSWD:` : aucune saisie de mot de passe requise (indispensable, car exécuté automatiquement sans interaction)

humaine) - Chaque ligne restreint l'autorisation à **un seul script précis**, par chemin absolu — aucune autre commande n'est autorisée par cette règle

13. Vérifications

Élément à vérifier	Commande de vérification	Résultat attendu
Connectivité IP brute	<code>ping -c 3 8.8.8.8</code>	0% de perte
Connectivité + DNS	<code>ping -c 3 google.com</code>	0% de perte
Interfaces réseau actives	<code>ip a</code>	Ligne <code>inet</code> présente sur chaque interface attendue
Service Asterisk actif	<code>sudo systemctl status asterisk --no-pager</code>	Active: active (running)
Comptes SIP enregistrés	<code>sudo asterisk -rx "pjsip show endpoints"</code>	<code>Not in use</code> pour chaque compte connecté
Cluster PostgreSQL actif	<code>pg_lsclusters</code>	Status: online
Module CDR chargé	<code>sudo asterisk -rx "cdr show status"</code>	<code>pgsql</code> listé dans "Registered Backends"
Table CDR alimentée	<code>sudo -u postgres psql -d asterisk_cdr -c "SELECT * FROM cdr;"</code>	Lignes correspondant aux appels passés
Liaison inter-bases fonctionnelle	<code>sudo -u postgres psql -d a2billing -c "SET ROLE a2billing_user; SELECT * FROM cdr_distant;"</code>	Mêmes lignes que la table <code>cdr</code> d'origine
Script de facturation exécutable et autorisé	<code>sudo -u asterisk sudo -u postgres /usr/local/bin/facturer.sh <ext> <sec></code>	Message Facturé X.XX Ar pour <ext>
Solde/dette mis à jour	<code>sudo -u postgres psql -d a2billing -c "SELECT * FROM comptes;"</code>	Valeurs cohérentes avec les appels/recharges effectués
Code de recharge supprimé après usage	<code>sudo -u postgres psql -d a2billing -c "SELECT * FROM codes_recharge;"</code>	Le code utilisé n'apparaît plus

14. Tests réalisés

Test	Méthode	Résultat
Appel IVR complet (8000 → message vocal)	Appel depuis MicroSIP (6001)	[FAIT] Message d'accueil entendu
Routage vers service commercial (touche 1)	Idem + touche 1	[FAIT] Redirection vers extension 100, appel décroché par 6002
Routage vers support technique (touche 2)	Idem + touche 2	[FAIT] Redirection vers extension 101
Routage vers comptabilité (touche 3)	Idem + touche 3	[FAIT] Redirection vers extension 102
Mise en attente (Hold)	Clic sur Hold en cours de communication, puis reprise	[FAIT] Musique d'attente entendue, reprise fonctionnelle
Transfert d'appel	Option <code>tT</code> activée sur <code>Dial()</code>	[FAIT] Configuré (fonctionnalité activée, non détaillée davantage dans l'historique)
Enregistrement CDR d'un appel réel	Appel complet + <code>SELECT * FROM cdr;</code>	[FAIT] Ligne enregistrée avec <code>src</code> , <code>dst</code> , durée, disposition corrects
Liaison <code>postgres_fdw</code>	<code>SELECT * FROM cdr_distant;</code> depuis <code>a2billing</code>	[FAIT] 3 lignes retrouvées, identiques à la base source
Facturation manuelle (fonction directe)	<code>SELECT facturer_appel('6001', 21);</code>	[FAIT] Solde déduit correctement (1,75 Ar pour 21 secondes à 5 Ar/min)
Facturation automatique après appel réel	Appel complet, puis vérification du solde sans action manuelle	[FAIT] Solde déduit automatiquement (confirmé à deux reprises avec des montants différents)
Facturation postpayée	<code>SELECT facturer_appel('6002', 120);</code>	[FAIT] Dette augmentée (0 → 50 Ar pour 120 secondes), solde resté à 0
Recharge manuelle (fonction directe)	<code>SELECT</code> <code>utiliser_code_recharge('6001', 'REC-AAA111');</code>	[FAIT] Solde augmenté de 10 000 Ar, code supprimé de la table
Recharge par composition téléphonique	Appel du code <code>#222#</code> puis <code>#444#</code> depuis un softphone	[FAIT] Message vocal de confirmation, dette/solde mis à jour, entrée créée dans <code>recharge.log</code>
Recharge sur compte postpayé	<code>SELECT</code> <code>utiliser_code_recharge('6002', '#444#');</code> (après correction de la fonction)	[FAIT] Dette diminuée de 30 à 10 (recharge de 20), solde resté à 0
Conférence à 2 participants	6001 et 6002 composent 9000	[FAIT] Connexion établie
Conférence à 3 participants	6001, 6002 et 6003 (MicroSIP portable) composent 9000	[FAIT] Les trois softphones affichent un statut "Connecté" simultanément sur la conférence

15. Conseils et bonnes pratiques

1. **Diagnostiquer couche par couche** : connectivité IP brute (`ping` sur une IP) avant DNS (`ping` sur un nom), puis service, puis port, puis pare-feu, puis logs applicatifs, puis syntaxe de configuration. Ne jamais sauter une étape en se fiant uniquement au message d'erreur affiché en premier.
2. **Toujours observer les logs en direct** (`asterisk -rvvv` , ou `tail` sur un fichier de log pendant l'action) au moment où le problème se produit, plutôt que d'essayer de le reproduire de mémoire après coup.
3. **Chercher le bon chemin avant de conclure à une absence de fichier** : utiliser `find / -name "..."` `2>/dev/null` ou `ls -la` sur le dossier parent plutôt que de supposer un chemin par convention.
4. **Revérifier systématiquement qu'un changement sensible a bien été enregistré**, particulièrement pour les fichiers de permissions (`sudoers`) où un échec de sauvegarde peut être silencieux.
5. **Ne jamais confondre “propriétaire d’une base” et “propriétaire d’une table”** en PostgreSQL — toujours vérifier avec `\dt` en cas de comportement inattendu autour des permissions.
6. **Isoler chaque maillon d’une chaîne complexe et le tester indépendamment** (ex. : tester un script manuellement avant de le brancher à Asterisk) plutôt que de tester uniquement le système complet et deviner où ça casse.
7. **Préférer un script dédié à une commande complexe intégrée directement dans une configuration** dès que des caractères spéciaux (virgules, points-virgules, guillemets) sont en jeu.
8. **En cas de solution logicielle non maintenue pour une technologie précise (ex. A2Billing + PostgreSQL), vérifier la réalité du support avant de s’engager dans une installation** — une simple mention dans la documentation générale ne garantit pas un support réel et à jour.
9. **Privilégier des applications indépendantes plutôt que le multi-compte d’une même application** lorsque plusieurs identités doivent rester actives simultanément pour des tests.
10. **Documenter les bugs au fil de l’eau plutôt qu’à la fin** — cela permet de mieux restituer le contexte exact (message d'erreur, hypothèse testée, cause réelle) et constitue un excellent matériau de préparation pour une soutenance orale.
11. **Ne jamais oublier les lignes `#include` lors d’une réinstallation complète du projet**. Asterisk ne lit que `pjsip.conf` et `extensions.conf` par défaut — les fichiers personnalisés (`pjsip_custom.conf` , `extensions_custom.conf`) doivent explicitement y être rattachés :

```
bash echo '#include pjsip_custom.conf' | sudo tee -a /etc/asterisk/pjsip.conf echo '#include extensions_custom.conf' | sudo tee -a /etc/asterisk/extensions.conf sudo asterisk -rx "core reload"
```

Piège à connaître : l'absence de ces lignes ne provoque **aucune erreur visible** au démarrage d'Asterisk — les comptes SIP définis dans le fichier personnalisé sont simplement absents de `pjsip show endpoints` , ce qui peut faire croire à tort à un problème de mot de passe ou de réseau. Vérifier systématiquement `pjsip show endpoints` après toute réinstallation, avant de chercher plus loin.
12. **En cas de réinstallation complète du projet sur une nouvelle VM, revérifier chaque chemin dépendant de la distribution** (dossier AGI, dossier des modules, dossier des logs) plutôt que de recopier des chemins d'une installation précédente — voir la méthode `grep astagidir` en section 7, Difficulté n°3.
13. **Un compte SIP créé n’est jamais automatiquement composable**. Déclarer `[6003]` dans `pjsip_custom.conf` définit *qui peut se connecter*, mais ne crée aucune règle permettant de le *joindre*. Il faut toujours une extension séparée dans le dialplan : `ini exten => 6003,1,Dial(PJSIP/6003,20)` Sans elle,

composer directement ce numéro renvoie `404 Not Found`, alors même que le compte apparaît correctement dans `pjsip show endpoints`.

14. **`psql -c "..."` n'interprète jamais les variables `-v`**, même avec la syntaxe `: 'nom'` — c'est une limitation documentée de PostgreSQL, pas une erreur de frappe. Seule la forme suivante fonctionne : `bash echo "SELECT ma_fonction(:'param');"` | `psql -d base -v param="$VALEUR"` Ce détail a fait échouer une première tentative de sécurisation anti-injection SQL dans ce projet avant d'être identifié.
 15. **Une extension de dialplan doit exister dans le même contexte que celui assigné aux comptes SIP** (`context=ivr-entree` dans `pjsip_custom.conf`). Une extension parfaitement écrite mais placée dans un autre contexte (ex. `[services]`) reste invisible pour un softphone qui compose directement — même symptôme `404 Not Found` que le point 13, cause différente (ici, mauvais emplacement plutôt qu'absence totale).
 16. **Un repli automatique vers un agent libre (`queue`) nécessite deux réglages combinés, pas un seul :**
 - `device_state_busy_at=1` sur chaque endpoint concerné (`pjsip_custom.conf`)
 - `ringinuse = no` dans la définition de la file (`queues.conf`)Oublier l'un des deux fait échouer silencieusement la détection d'occupation, sans message d'erreur — seul un test avec `queue show <nom>` pendant un appel actif permet de le confirmer.
 17. **`pg_lsclusters` est la commande de référence pour vérifier l'état réel de PostgreSQL**, plus fiable que `systemctl status postgresql` qui affiche `active (exited)` même quand tout fonctionne normalement (ce statut signifie "le service de démarrage a fini sa tâche", pas "le service est arrêté").
 18. **Après un `systemctl edit <service>` ou la création manuelle d'un override, vérifier que la ligne `Drop-In:` apparaît bien** dans `systemctl status <service> --no-pager` — sa présence confirme que le fichier de surcharge est réellement pris en compte, pas seulement qu'il existe sur le disque.
-

16. Mise à jour — Session complémentaire (repli d'agents, extensions directes, sécurisation)

Cette section documente une seconde phase de travail, réalisée après la première version de ce document, en conservant le même niveau de détail et la même exigence de statut [FAIT]/[PARTIEL]/[NON FAIT]/[EN COURS] que le reste du projet.

16.1 Correction de sécurité — injection SQL dans les scripts de facturation [EN COURS]

**** Objectif : **** éliminer le risque d'injection SQL dans `facturer.sh` et `recharger.sh`, qui construisaient leurs requêtes en collant directement du texte reçu en argument.

**** Concept : **** au lieu d'insérer le texte brut dans la commande SQL (risque : un caractère `'` suivi de SQL arbitraire pourrait casser ou détourner la requête), on utilise les **variables** `psql` (`-v nom=valeur` puis `:'nom'` dans la requête), qui échappent automatiquement toute valeur comme une chaîne SQL littérale.

**** Configuration corrigée : ****

```
# facturer.sh
psql -d a2billing -v ext="$EXT" -v sec="$SEC" -c "SELECT facturer_appel(:'ext', : 'sec':integer);"
```

```
# recharger.sh
psql -d a2billing -v ext="$EXT" -v code="$CODE" -c "SELECT utiliser_code_recharge(:'ext', : 'code');"
```

Statut : [PARTIEL] Le code corrigé a été fourni et expliqué, mais **n'a pas encore été réappliqué et retesté sur la VM** dans l'historique de cette conversation — à faire avant le rendu final, en réutilisant le protocole de test déjà connu (`sudo -u asterisk sudo -u postgres /usr/local/bin/facturer.sh <ext> <sec>` puis vérification de `SELECT * FROM comptes;`).

16.2 Extensions d'appel direct manquantes (6001, 6002, 6003) [FAIT]

**** Objectif : **** permettre à un compte SIP d'en appeler directement un autre par son numéro (ex : composer `6003` depuis `6002`), fonctionnalité jusque-là absente.

**** Concept : **** un compte SIP (défini dans `pjsip_custom.conf`) et une extension composable (définie dans `extensions_custom.conf`) sont deux choses distinctes dans Asterisk — le premier définit *qui peut se connecter*, le second définit *ce qui se passe quand on compose un numéro*. Avoir un compte enregistré ne crée aucune règle automatique de routage vers lui.

**** Architecture : **** ajout de règles dans le contexte `[ivr-entree]`, celui utilisé par tous les softphones.

**** Configuration appliquée : ****

```
exten => 6001,1,Dial(PJSIP/6001,20)
exten => 6002,1,Dial(PJSIP/6002,20)
exten => 6003,1,Dial(PJSIP/6003,20)
```

[FAIT] Test : composition directe de 6003 depuis 6002 — appel abouti (auparavant : 404 Not Found — Numérotation incorrecte ou numéro inexistant).

**** À retenir :** question d'examen probable — *“pourquoi je ne pouvais pas appeler un compte SIP pourtant bien enregistré ?”* Réponse : absence de règle de dialplan associée, un compte SIP seul ne suffit pas.

16.3 Repli automatique vers un second agent si le premier est occupé [FAIT]

**** Objectif :** répondre à l'exigence “il faut qu'il y ait un autre poste qui prenne l'appel si l'un est occupé”.

**** Concept :** `Dial(A&B)` fait sonner plusieurs postes en même temps mais **sans vérifier leur disponibilité** — un poste déjà en communication continue de sonner pour de nouveaux appels. La vraie brique adaptée est la **file d'attente** (`Queue`), combinée à la détection d'occupation (`device_state_busy_at`).

**** Architecture :**

```
Appel → Extension service → Queue(service_client) → agents libres uniquement (6002, 6003)
```

**** Configuration appliquée :**

Dans `pjsip_custom.conf` , ajout dans les blocs `type=endpoint` de 6002 et 6003 :

```
device_state_busy_at=1
```

Nouveau fichier `/etc/asterisk/queues.conf` :

```
[service_client]
strategy = ringall
timeout = 20
ringinuse = no
member => PJSIP/6002
member => PJSIP/6003
```

Dans `extensions_custom.conf` , remplacement des `Dial(PJSIP/6002,20,tT)` par :

```
same => n,Queue(service_client,t,,30)
```

**** Bug rencontré** — `ringinuse` non désactivé au premier essai - Symptôme : `6002` restait marqué `Not in use` par `queue show` alors qu'il était en communication active - **Cause** : `ringinuse = no` n'avait pas encore été ajouté à `queues.conf` lors du premier test - **Solution** : ajout de la ligne, puis `sudo asterisk -rx "module reload app_queue.so"`

**** Bug rencontré — faux négatif de test dû à un mauvais protocole - Symptôme :** ****** aucun repli observé lors d'un premier essai - **Cause réelle :** le test n'isolait pas correctement la variable observée — c'était 6002 lui-même qui initiait l'appel testé vers 8000, un cas d'auto-appel sans rapport avec la disponibilité réelle du poste - **Solution :** protocole corrigé — occuper un agent via un appel **séparé** (ex : 6003 appelle 6002, sans raccrocher), puis tester depuis un troisième compte (6001), non impliqué dans l'appel d'occupation

**** Bug rencontré — décalage de timing dans la vérification - Symptôme :** ****** `sudo asterisk -rx "core show channels"` affichait 0 active channels alors qu'un appel semblait actif à l'écran - **Cause :** les appels s'étaient terminés entre le moment observé à l'écran et le moment où la commande de vérification a été tapée - **Solution :** utilisation de `watch -n 2 'sudo asterisk -rx "core show channels"'` pour une observation continue en direct, évitant tout décalage

**** Bug rencontré — watch lancé sans sudo à l'intérieur - Message exact :** ****** `Unable to connect to remote asterisk (does /var/run/asterisk/asterisk.ctl exist?)` - **Cause :** `sudo` doit être placé à l'intérieur des guillemets de la commande répétée par `watch`, pas avant `watch` lui-même - **Solution :** `watch -n 2 'sudo asterisk -rx "core show channels"'`

[FAIT] Test final réussi : 1. 6002 appelle 8000/1 → seul 6003 (libre) sonne, décroche 2. 6001 appelle 8000/1 pendant que 6002 et 6003 sont tous deux occupés → musique d'attente, puis abandon après le timeout de 20 secondes (comportement correct : aucun agent disponible) 3. Dès que 6002 et 6003 se libèrent simultanément → les deux sonnent en même temps pour l'appel de 6001 resté en file — comportement cohérent avec la stratégie `ringall` ("sonner tous les agents libres")

**** À retenir :** `ringall` = tous les agents libres sonnent simultanément, premier arrivé décroche (rapide, mais répartition potentiellement inégale). Alternative documentée mais **non appliquée** : `strategy = leastrecent`, qui ferait sonner en priorité l'agent resté libre le plus longtemps sans appel (répartition plus équitable, légèrement plus lente). Statut : [NON FAIT] (uniquement expliquée, la configuration active du projet reste `ringall`).

16.4 Clarification — conférence par numéro dédié vs fusion côté softphone

**** Concept clarifié :** ****** deux mécanismes distincts permettent d'obtenir une conférence à 3 : - **Côté serveur**, via un numéro dédié (9000 → `ConfBridge`) : Asterisk mixe lui-même l'audio de tous les participants dans une salle centralisée — [FAIT] validé (voir section 6, test conférence à 3 participants) - **Côté client**, via les fonctions "Add call" + "CONF"/"Merge" du softphone (ex : 6001 appelle 6003, puis ajoute 6002) : c'est le softphone qui fusionne localement deux appels distincts, sans configuration serveur supplémentaire — [NON FAIT] non testé dans l'historique du projet, seulement expliqué (limitation possible : fonctionnalité parfois réservée aux versions payantes de certains softphones, comme observé pour le DND de Zoiper)

16.5 Décisions techniques complémentaires

Décision	Justification
<code>Queue()</code> plutôt que <code>Dial(A&B)</code> pour le repli d'agent	<code>Dial(A&B)</code> ne vérifie pas la disponibilité réelle des postes ; <code>Queue()</code> combinée à <code>device_state_busy_at</code> exclut les agents déjà occupés, répond précisément à l'exigence du prof
Variables <code>psql -v</code> plutôt que concaténation de texte dans les scripts de facturation	Élimine le risque d'injection SQL, bonne pratique à mentionner spontanément à l'oral plutôt que de risquer une question piège dessus
Conservation de <code>ringall</code> plutôt que passage à <code>leastrecent</code>	Aucune exigence explicite du cahier des charges sur l'équité de répartition entre agents ; <code>ringall</code> déjà testé et validé, <code>leastrecent</code> documenté comme amélioration possible plutôt qu'appliqué sans nécessité avérée

17. Mise à jour — Session 3 (contrôle de crédit temps réel, fiabilisation démarrage, console web, HTTPS)

17.1 Bug — CDR non enregistré après redémarrage de la VM [EN COURS]→[FAIT]

**** Contexte :** après une longue pause de travail (reprise le 20-08-2026), un appel test n'apparaissait ni sur la console web ni dans la base `asterisk_cdr`.

Message d'erreur exact (observé en direct via `asterisk -rvvv`) :

```
ERROR: cdr_pgsql.c:434 pgsql_log: Failed to insert call detail record into database!
Reason: ERROR: syntax error at or near ")"
LINE 1: INSERT INTO cdr () VALUES ()
```

**** Analyse :** symptôme identique à un bug déjà rencontré (propriétaire de table incorrect), mais cette fois `\dt` a confirmé que `asterisk_cdr` restait bien propriétaire de la table, et `\d cdr` a confirmé une structure intacte — donc une cause différente pour un symptôme identique.

**** Cause réelle :** au démarrage de la VM, Asterisk a démarré **avant** que PostgreSQL soit complètement prêt à accepter des connexions. Le module `cdr_pgsql` a alors mis en cache une liste de colonnes vide au moment de sa première tentative de connexion, et n'a jamais retenté de la relire ensuite.

**** Solution immédiate (déblocage) :**

```
sudo asterisk -rx "module unload cdr_pgsql.so"
sudo asterisk -rx "module load cdr_pgsql.so"
```

[FAIT] Confirmé : un nouvel appel s'enregistre immédiatement après ce rechargement.

**** Solution durable (empêcher la récurrence au prochain redémarrage) :**

```
sudo mkdir -p /etc/systemd/system/asterisk.service.d
sudo nano /etc/systemd/system/asterisk.service.d/override.conf
```

```
[Unit]
After=postgresql.service
Wants=postgresql.service
```

```
sudo systemctl daemon-reload
```

[FAIT] Test de non-régression (le plus rigoureux du projet) : redémarrage complet de la VM (`sudo reboot`), puis vérification post-redémarrage :

```
sudo systemctl status asterisk --no-pager    # → active (running), Drop-In: override.conf listé
sudo asterisk -rx "cdr show status"          # → pgsql toujours dans les Registered Backends
```

Un appel passé juste après ce redémarrage s'est enregistré **sans aucune intervention manuelle** — confirmation que le correctif est bien permanent, pas un simple contournement ponctuel.

**** À retenir :** un même symptôme (`INSERT INTO cdr ( ) VALUES ( )`) peut avoir deux causes complètement différentes (propriétaire de table incorrect vs ordre de démarrage des services) — d'où l'importance de revérifier chaque hypothèse plutôt que de réappliquer par réflexe la dernière solution trouvée pour un symptôme qui ressemble à un bug déjà connu.

17.2 Contrôle de crédit prépayé en temps réel (blocage avant appel) [FAIT]

**** Objectif :** empêcher un compte prépayé à solde ≤ 0 de passer un appel — jusqu'ici, la facturation ne se faisait qu'après l'appel (voir section 17.2 bis ci-dessous pour la distinction), ce qui permettait à un compte à 0 d'appeler quand même et de passer en négatif.

**** Concept :** ajout d'une vérification **avant** le `Dial()` / `Queue()`, via la fonction Asterisk `${SHELL(...)}`, qui exécute une commande shell et récupère sa sortie texte directement dans une variable du dialplan — contrairement à `System()` qui n'attend aucun retour.

**** Architecture :**

```
Answer() → ${SHELL(verifier_credit.sh)} → GotoIf(REFUS ? raccroche : continue) → Queue()
```

**** Configuration :**

Nouveau script `/usr/local/bin/verifier_credit.sh` :

```
#!/bin/bash
EXT="$1"
echo "SELECT CASE WHEN mode='prepaid' AND solde<=0 THEN 'REFUS' ELSE 'OK' END FROM comptes WHERE extension=:ext";" \
| psql -d a2billing -v ext="$EXT" -tA 2>/dev/null | tr -d '[:space:]'
```

- `-tA` : sortie “tuples only” non alignée → uniquement `OK` ou `REFUS`, sans en-tête
- `tr -d '[:space:]'` : supprime tout espace/retour à la ligne résiduel, indispensable pour que le `GotoIf` compare des chaînes strictement identiques

Ajout dans `/etc/sudoers.d/asterisk-billing` :

```
asterisk ALL=(postgres) NOPASSWD: /usr/local/bin/verifier_credit.sh
```

Dans `extensions_custom.conf`, chaque service (`100`, `101`, `102`) suit désormais ce modèle :

```

exten => 100,1,Answer()
same => n,Set(CHANNEL(hangup_handler_push)=facturation,h,1)
same => n,Set(CREDIT=${SHELL(sudo -u postgres /usr/local/bin/verifier_credit.sh ${CALLERID(num)}})}
same => n,GotoIf("${CREDIT}" = "REFUS"?refuse:continue)
same => n(refuse),agi(googletts.agi,"Solde insuffisant. Veuillez recharger votre compte avant de reessayer.",fr)
same => n,Hangup()
same => n(continue),agi(googletts.agi,"Mise en relation avec le service commercial.",fr)
same => n,Queue(service_client,t,,30)
same => n,Hangup()

```

[FAIT] Tests réalisés (dans l'ordre) : 1. Solde de 6001 forcé à 0 → `verifier_credit.sh` testé isolément → retourne REFUS 2. Appel réel 8000 → 1 depuis 6001 à solde 0 → message vocal "Solde insuffisant..." entendu, `Hangup()` immédiat, **aucune sonnerie côté 6002/6003** 3. Solde repositionné à un montant positif → même appel → comportement normal, sonnerie et communication établies

**** À retenir — distinction essentielle pour l'oral : ****

	Facturation (déjà en place depuis la session précédente)	Contrôle de crédit (ajouté ici)
Se déclenche	Après l'appel (hangup handler)	Avant l'appel (dès l'entrée dans le service)
Rôle	Constater et déduire le coût réel	Autoriser ou refuser l'appel
Équivalent opérateur télécom	Facturation classique / CDR	<i>Online Charging System (OCS)</i> , souvent lié à la norme CAMEL en télécom mobile prépayé

Avant cet ajout, un compte prépayé à 0 pouvait passer un appel et finir en solde négatif — c'était une limite assumée et documentée (section 16). Elle est maintenant corrigée et vérifiée.

17.3 Console web de supervision (comptes + CDR) [FAIT]

**** Objectif : **** afficher dans un navigateur, sans commande SQL, les comptes (solde/dette) et les derniers appels — décision explicite du projet : **cette fonctionnalité n'est pas intégrée en continu au projet Asterisk lui-même**, elle est développée et testée séparément, dans l'optique de peaufiner un support de présentation/CV après la soutenance.

**** Architecture : ****

```

Navigateur → Apache + PHP (PDO pgsql) → PostgreSQL (a2billing + asterisk_cdr)

```

**** Composants : **** `apache2`, `php`, `php-pgsql`, fichier unique `/var/www/html/index.php` (connexion en lecture seule aux deux bases, cartes de comptes avec code couleur solde/dette, tableau des 25 derniers appels CDR, rafraîchissement automatique toutes les 5 secondes).

[FAIT] Test : page accessible à `http://192.168.56.101/`, comptes et appels affichés correctement, mise à jour automatique confirmée après un appel réel.

** À retenir : ** interface strictement **read-only** — aucune action (facturation, recharge) n'est possible depuis la console, seulement depuis le dialplan/scripts existants. Choix assumé de séparation claire entre outil de consultation et outils d'administration, cohérent avec les pratiques professionnelles réelles (voir comparaison section suivante).

17.4 HTTPS + nom de domaine local pour la console [EN COURS] (en cours)

** Objectif : ** servir la console via `https://voip.local` plutôt que `http://192.168.56.101` .

** Concept clé, à bien distinguer : ** un certificat public (Let's Encrypt) nécessite un domaine réellement accessible depuis Internet pour la vérification — **impossible** sur ce projet, la VM n'étant joignable qu'en réseau privé (Host-only). Solution retenue : un **certificat auto-signé** (chiffrement TLS réel, mais non authentifié par une autorité publique — le navigateur affiche donc un avertissement à accepter manuellement) combiné à une résolution de nom **locale**, via le fichier `hosts` du PC (pas un vrai enregistrement de domaine public).

** Configuration fournie : ** - Génération d'un certificat auto-signé avec `openssl req -x509` (validité 365 jours, `CN=voip.local`) - Activation du module SSL Apache (`a2enmod ssl`) - Nouveau site `/etc/apache2/sites-available/voip-ssl.conf` avec `SSLEngine on` sur le port 443 - Ajout de `192.168.56.101 voip.local` dans le fichier `hosts` du PC hôte (pas de la VM)

Statut : [FAIT] — testé et confirmé : `https://voip.local` s'affiche correctement depuis le PC, après acceptation manuelle de l'avertissement de certificat auto-signé.

** À retenir pour l'oral : ** savoir expliquer *pourquoi* le cadenas n'est pas vert est plus important que d'essayer d'obtenir un vrai certificat impossible à obtenir ici — la distinction entre **chiffrement** (assuré, même en auto-signé) et **authentification publique par une autorité reconnue** (impossible en réseau privé) est le vrai point de compréhension attendu.

17.5 Bug — résolution du nom local `voip.local` non enregistrée [FAIT]

** Contexte : ** après avoir changé de source Internet (basculer sur le partage de connexion d'un téléphone pour la présentation), `https://voip.local` a cessé de répondre avec le message navigateur *"Désolé, impossible d'accéder à cette page."*

** Diagnostic, couche par couche : 1. `ping 192.168.56.101` depuis le PC → [FAIT] répond (le réseau Host-only n'est pas en cause) 2. `http://192.168.56.101/` dans le navigateur → [FAIT] fonctionne (Apache tourne, le service web n'est pas en cause) 3. `ping voip.local` depuis le PC** → La requête Ping n'a pas pu trouver l'hôte `voip.local`

** Cause réelle : ** le fichier `C:\Windows\System32\drivers\etc\hosts` du PC ne contenait **jamais** la ligne `192.168.56.101 voip.local` — la modification effectuée lors de la configuration initiale du HTTPS n'avait pas été réellement enregistrée. Cause la plus probable : le Bloc-notes avait été ouvert **sans droits administrateur**, ce qui permet de modifier le contenu à l'écran mais bloque silencieusement l'enregistrement sur ce fichier système protégé — exactement le même type de piège silencieux que celui rencontré avec `visudo` (section 16.3), mais côté Windows cette fois.

** Solution : ** ajout de la ligne via PowerShell **exécuté en tant qu'administrateur**, méthode plus fiable que l'édition manuelle au Bloc-notes :

```
Add-Content -Path C:\Windows\System32\drivers\etc\hosts -Value "192.168.56.101` tvoip.local"
```

[FAIT] Test : `type C:\Windows\System32\drivers\etc\hosts` confirme la ligne ajoutée ; `ping voip.local` répond depuis `192.168.56.101` ; `https://voip.local` affiche la console de supervision normalement (avertissement de certificat auto-signé toujours présent, comportement attendu et déjà documenté en 17.4).

**** À retenir :** un changement de réseau (Wi-Fi → partage de connexion) ne modifie ni le fichier `hosts` , ni la configuration Host-only de VirtualBox — ce sont deux éléments totalement indépendants d'Internet. Le vrai enseignement ici est plus général : **toute modification d'un fichier système protégé doit être vérifiée par une relecture immédiate du fichier, jamais supposée réussie sur la seule foi de l'interface d'édition** — un fichier peut sembler modifié à l'écran sans que l'enregistrement ait réellement eu lieu, silencieusement, faute de droits suffisants.

17.5 Bug — résolution de `voip.local` perdue avant la présentation [ENCOURS]→[FAIT]

**** Contexte :** en préparant un test de bascule réseau (utilisation du téléphone comme point d'accès Internet pour la présentation), `https://voip.local` a cessé de répondre côté navigateur (« Désolé, impossible d'accéder à cette page »).

**** Diagnostic, étape par étape :** 1. `ping 192.168.56.101` depuis le PC → répond normalement (réseau Host-only intact) 2. `http://192.168.56.101/` en HTTP simple → répond normalement (Apache fonctionne, donc le problème n'est ni réseau, ni serveur) 3. `ping voip.local` → testé une première fois par erreur dans le terminal SSH de la VM** (donc sans rapport avec le vrai problème, qui concerne la résolution de noms côté PC, pas côté VM) → Temporary failure in name resolution 4. `ping voip.local` refait au bon endroit, **dans PowerShell sur le PC** → La requête Ping n'a pas pu trouver l'hôte `voip.local` 5. `type C:\Windows\System32\drivers\etc\hosts` → la ligne `192.168.56.101 voip.local` **n'était pas présente**, alors qu'elle avait pourtant été ajoutée lors de la configuration initiale du HTTPS (section 17.4)

**** Cause probable :** l'édition initiale du fichier `hosts` via le Bloc-notes n'a probablement pas été sauvegardée réellement, faute d'avoir ouvert l'éditeur avec les droits administrateur — Windows protège ce fichier système et peut silencieusement refuser ou rediriger l'écriture sans avertissement clair pour l'utilisateur, un peu comme `visudo` qui refuse d'enregistrer un fichier `sudoers` en cas d'erreur (bug déjà rencontré en section 7, Difficulté n°10).

**** Solution appliquée :**

```
# Dans PowerShell ouvert en tant qu'administrateur
Add-Content -Path C:\Windows\System32\drivers\etc\hosts -Value "192.168.56.101` tvoip.local"
```

[FAIT] Test : `type` du fichier confirme la ligne ajoutée ; `https://voip.local` de nouveau accessible (avertissement de certificat auto-signé, comme attendu) ; console affichant correctement comptes et CDR.

**** À retenir :** toute modification d'un fichier protégé par le système d'exploitation (`/etc/sudoers.d/` sous Linux, `hosts` sous Windows) doit être **révérifiée immédiatement après édition**, jamais supposée réussie simplement parce que l'éditeur ne montre pas d'erreur visible à l'écran — c'est la même leçon apprise deux fois, sur deux systèmes différents, au cours de ce projet.

17.6 Portabilité réseau — présentation via partage de connexion téléphone [FAIT] (analyse)

**** Question posée :** l'architecture réseau du projet doit-elle être reconfigurée si le PC utilise le partage de connexion d'un téléphone plutôt que le Wi-Fi habituel pour accéder à Internet le jour de la présentation ?

**** Analyse :** non, aucune reconfiguration n'est nécessaire. L'adaptateur **NAT** de VirtualBox route automatiquement à travers la connexion Internet active du PC, quelle qu'elle soit (Wi-Fi, Ethernet, partage de connexion) — c'est transparent. L'adaptateur **Host-only**, lui, ne dépend jamais d'Internet : c'est un réseau privé virtuel géré uniquement par VirtualBox. Or c'est justement cet adaptateur Host-only qui porte tout ce qui compte pour la démonstration (softphones, console web).

**** À retenir :** seule l'installation initiale des paquets et la synthèse vocale Google TTS nécessitent une vraie connexion Internet — le cœur fonctionnel du projet (Asterisk, PostgreSQL, facturation, console) fonctionne entièrement en local, indépendamment de la source d'accès à Internet du PC hôte.

18. Conclusion

Ce projet a permis de construire, de bout en bout, un système IPBX fonctionnel couvrant l'ensemble des exigences du cahier des charges initial : accueil vocal automatisé par synthèse vocale, routage intelligent des appels vers plusieurs services, fonctionnalités d'appel avancées (mise en attente, transfert, conférence multi-participants), journalisation systématique des appels dans une base de données dédiée, et un système de facturation complet distinguant les modes prépayé et postpayé, avec gestion de codes de recharge générés, utilisés et supprimés automatiquement.

Au-delà du résultat technique final, le parcours du projet a été marqué par une quinzaine de blocages distincts, chacun résolu par une démarche de diagnostic méthodique — observation des logs en temps réel, isolement des causes couche par couche, vérification systématique après chaque correction plutôt que supposition. Cette méthode, plus que les commandes elles-mêmes, constitue l'apprentissage central du projet et une compétence directement transférable à tout contexte d'administration système ou de support technique.

Une décision technique notable mérite d'être soulignée : plutôt que d'installer le logiciel A2Billing officiel — incompatible avec l'exigence de deux bases PostgreSQL distinctes formulée dans le cahier des charges — le choix a été fait de reconstruire la logique de facturation demandée directement en PL/pgSQL. Ce choix, documenté et assumé, a permis de rester fidèle à la fois à la consigne technique et à l'esprit pédagogique du projet, chaque mécanisme de facturation étant entièrement compris et maîtrisé plutôt qu'installé en boîte noire.